

EVOLUTIONARY PROGRAMMING AND GENETIC ALGORITHMS — SUPREME EXAM MANUAL V3

What changed in this Supreme V3

This version uses **NEW_MASTER_MANUAL** as the accuracy backbone, expands it with the best exam-drill features from **exam_improved.md**, and now adds the strongest material from **exam_final.md**. The result is a single master document that combines:

- safe theory and exam-risk notes from the master manual;
- frequency labels, answer-pattern cues and MCQ recognition language from the improved edition;
- end-of-chapter exercises and calculation drills;
- corrected wording where the improved edition was too absolute;
- a final memorization and operator-calculation workshop;
- a new final-edition quick dashboard, per-chapter exam handles, and a V3 risk matrix.

Editorial rule used in this merge

When a previous-exam answer pattern conflicts with stable theory, this manual separates:

1. **Theory-safe answer** — what is conceptually correct according to EA/GA theory and course definitions.
2. **Exam-risk answer** — what a previous MCQ key appears to expect.
3. **Safe exam behavior** — how to answer when the wording is ambiguous.

This matters because the MCQ corpus contains repeated, near-repeated and sometimes inconsistent keys. Do not memorize a suspicious key without reading the relevant Exam-risk note.

Master chapter map and frequency guide

Chapter	Topic	MCQ Answer Key	Frequency
1	Evolutionary Computation Foundations	—	High
2	Problem Classes Solved by EC	1, 4, 6	Very High
3	General EA Structure and Components	1,3,5,7,10,12,14,15	Very High
4	Representation, Genotype, Phenotype	2, 5 (depends + influences)	High
5	Main GA Representations	Varies by type	High
6	Fitness/Evaluation Function	9 or 2,4	Very High
7	Population, Diversity, Init, Termination	c,e,f / a,b	High
8	Hillclimbing	1,2,5,7	High
9	Mutation Operators	Varies	Medium
10	Recombination/Crossover	e,f,g,h,j	High

11	Permutation Crossover (PMX, OX, CX, ECX)	Calculation	Very High
12	Parent Selection Mechanisms	Varies	High
13	Survivor Selection	b,d,e,f,k	High
14	Classical Genetic Algorithm Model	1,3,5,8	Very High
15	Applications and Representation	Varies	Medium
16	Evolution Strategies (ES, ES-2M)	d (2,4,5,6)	High
17	Memetic and Hybrid Algorithms	d (2,4,6,7,9)	Medium
18	Exam Traps and Final Mastery Rules	—	—
19	Operator Calculation Workshop	Worked examples	Very High
20	Final Coverage Checklist + Memorization	—	—

Global high-yield corrections before memorization

1. **Initial population.** It is generated before evolution begins, usually randomly and often uniformly. In constrained problems, feasibility may be enforced by representation, repair, filtering, penalties or survivor selection. Do not overgeneralize that all randomly generated individuals are automatically feasible.
2. **Survivor selection.** It may use the current population, descendants, or both. It may be deterministic, stochastic or mixed. It may use quality, age, or both. It does not universally guarantee higher average quality.
3. **Fitness.** Fitness evaluates each candidate/individual. It does not evaluate algorithm speed, resources, the whole population against previous generations, or select individuals by itself.
4. **Permutation representation.** A valid permutation has no duplicate elements and no missing elements. It requires special mutation and recombination operators.
5. **Canonical GA.** The safe reference model is binary representation, fitness-proportionate parent selection, high crossover probability and low mutation probability.
6. **Evolution Strategies.** Mutation is central and typically applied with probability 1; representation is commonly real-valued; self-adaptation often evolves mutation parameters.
7. **Operator calculations.** Always check validity first: length, domain, duplicates, missing values, and whether the operator preserves what the question says it preserves.

Supreme V3 final-edition quick dashboard

This dashboard is imported and corrected from **exam_final.md**. It is meant for the last review pass, not as a substitute for the theory chapters.

Compact answer-pattern table

Chapter	Topic	MCQ Answer Pattern	Frequency
1	EC Inspiration	Biological evolution	High
2	Problem Classes	1, 4, 6	Very High
3	EA Components	1,3,5,7,10,12,14,15	Very High
4	Representation Choice	2, 5 (depends + influences)	High

5	GA Representations	Varies by type	High
6	Fitness Function	9 or 2,4	Very High
7	Initial Population	Before evolution, random, uniform	High
7	Termination	Mandatory + iteration control	High
8	Hillclimbing	1, 2, 5, 7	High
10	Recombination	e,f,g,h,j	High
11	Permutation Crossover	Calculation	Very High
12	Parent Selection	Varies	High
13	Survivor Selection	b,d,e,f,k	High
14	Canonical GA	1, 3, 5, 8	Very High
15	Applications	Varies	Medium
16	Evolution Strategies	d (2,4,5,6 for ES-2M)	High
17	Memetic Algorithms	d (2,4,6,7,9)	Medium
19	Operator Calculations	Worked examples	Very High

V3 use protocol

1. **First pass:** read Chapters 1-17 for theory and worked examples.
2. **Second pass:** read Chapters 18-20 as the final filter and memorize the recurring answer patterns.
3. **Calculation pass:** hand-solve PMX, OX/OCX, CX, mutation-by-scramble, inversion, FPS, SUS, and ES sigma update.
4. **Trap pass:** for every MCQ, ask whether the statement describes the candidate, the population, the algorithm, the parent pool, the offspring, or the next generation.
5. **Corrupted-key pass:** if a key conflicts with theory, mark it as an exam-risk pattern rather than forcing it into theory.

Ultra-high-yield priority order

1. Problem classes: optimization, modeling, simulation.
2. EA components: representation, fitness, population, parent selection, variation, survivor selection, initialization, termination.
3. Fitness: evaluates each candidate/individual, not the algorithm.
4. Representation: depends on problem, affects quality, determines valid operators.
5. Permutations: no duplicates, special operators, PMX/OX/CX/ECX.
6. Selection: FPS, SUS, tournament, rank, parent vs survivor.
7. Canonical GA: binary, FPS, high crossover, low mutation.
8. ES: real vectors, mutation-centered, probability 1, self-adaptation.
9. Hillclimbing: one current point, local optimum, not guaranteed global optimum.
10. Final risk notes: survivor selection, initial population, TSP/PMX/ECX, corrupted CX keys.

CHAPTER 1 — Evolutionary Computation foundations

Final-edition quick exam handle

Exam frequency: High. Core answer: EC is inspired by natural biological evolution. Individual is static; population evolves.

Evolutionary Computation (EC, *calcul evolutiv*) is an informatics and computational-intelligence domain inspired by natural biological evolution (*evoluția naturală biologică*). In nature, evolution behaves like a large-scale generate-test or trial-and-error process: many individuals are produced, they differ by inherited and mutated traits, they compete in a limited-resource environment, and those that are better adapted are more likely to reproduce. The course maps this biological idea into problem solving: a population of candidate solutions is repeatedly varied and selected until sufficiently good solutions are found.

The important biological terms are not decorative; they become the algorithmic vocabulary. A limited-resource environment means the population cannot grow without bound, so there is competition. Reproduction corresponds to recombination/crossover (*recombinare / încrucișare*). Occasional mutation introduces new genetic material. Fitness (*adaptare / fitness*) expresses how well an individual fits the environment. Selection (*selecție*) favors better individuals, but most evolutionary algorithms keep some stochastic component so that search does not become a rigid deterministic climb.

Main EC families tested in the course are:

- **Genetic Algorithms (GA, algoritmi genetici)**: usually finite-alphabet strings, classical form with binary chromosomes, high crossover probability and low mutation probability.
- **Evolutionary Strategies (ES, strategii evolutive)**: usually real-number vectors, mutation-centered, Gaussian perturbation, and self-adaptation of mutation step sizes.
- **Evolutionary Programming (EP, programare evolutivă)**: historically associated with evolution of behavioral structures such as finite-state machines.
- **Genetic Programming (GP, programare genetică)**: evolves programs or expression trees.
- **Swarm Intelligence (inteligență de roi)**: population-like collective methods such as PSO and Firefly.

The central mapping is:

Biological/evolution term	EA/problem-solving term
Environment (<i>mediu</i>)	search/solution space (<i>spațiu de căutare / spațiul soluțiilor</i>)
Individual / chromosome / genotype	candidate solution encoded for the algorithm
Phenotype	decoded solution in the real problem space
Fitness/adaptation	quality/objective value used to guide search
Population	multiset of candidate genotypes
Reproduction/crossover	recombination of parent genotypes
Mutation	random local change of genetic material
Selection	mechanism favoring better candidates

An exam-safe formulation is: **an individual is a candidate solution; the population evolves; fitness measures the quality of each candidate; selection mechanisms use fitness to bias reproduction and survival.** The exam often gives false statements saying fitness evaluates the algorithm, measures speed/resource consumption, or is randomly changed every iteration. Those are false.

Supreme memory card

- EC inspiration = natural biological evolution, never industrial/information/agricultural revolution.
- Evolution is generate-test/trial-and-error under limited resources.
- Individual is static; population evolves/adapts.
- Families: GA, ES, GP, EP, swarm intelligence.

High-yield MCQ overlay from the improved edition

Exam Frequency: High — foundational concept, appears every session

Exam-risk note: Some MCQ versions spell it "evolutia naturala biologica" — this is ALWAYS the correct answer regardless of spelling.

- **Key elements:**

Exam trap: "Individul evolueaza pentru a se adapta mediului" — **FALSE**.

Practice exercises and drills

Exercise 1.1 — Concept Mapping: Given these biological elements (reproduction, mutation, competition, limited resources, fitness), explain in your own words how each maps to a computational concept in EC. Why is "limited resources" essential? What happens if resources are unlimited?

Exercise 1.2 — Family Identification: For each scenario below, identify which EC family is most appropriate and explain why:

- (a) Evolving a mathematical expression to fit data
- (b) Finding optimal parameters for a chemical process
- (c) Evolving behavior rules for a robot
- (d) Optimizing delivery routes for a fleet of trucks

Exercise 1.3 — True/False with Reasoning: Mark each statement as TRUE or FALSE and explain WHY:

- (a) The individual evolves over generations to become better adapted.
- (b) The population is the unit of evolution.
- (c) EC is inspired by the industrial revolution's optimization methods.
- (d) Swarm intelligence is part of the EC family.

Exercise 1.4 — Application Design: You are tasked with optimizing the layout of a warehouse (placing items to minimize picking time). What is the "environment," the "individual," the "fitness"? Is this a unimodal or multimodal problem? Justify.

CHAPTER 2 — Problem classes solved by Evolutionary Computation

Final-edition quick exam handle

Exam frequency: Very high. Answer pattern: 1,4,6 = optimization, modeling/system identification, simulation. Distractors: big data, validation, memory allocation, autonomous vehicles.

The course uses the classic model:

inputs -> model -> outputs

Three problem classes are explicitly tested.

1. Optimization problems (*probleme de optimizare*)

Known: the model and desired output or objective. Must compute: the inputs/decision variables that produce the best output.

Examples: Traveling Salesman Problem (TSP, *problema comis-voiajorului*), Job-Shop Scheduling (JSS, *planificarea activităților*), N-Queens (*problema celor N regine*), knapsack (*rucsac*), transportation/resource-allocation problems, portfolio optimization. Optimization asks: "Which solution maximizes/minimizes the objective while respecting constraints?"

2. Modeling / system identification (*probleme de modelare / identificarea sistemului*)

Known: inputs and outputs. Must compute: the model that explains or generalizes the input-output relationship.

Examples: supervised classification, machine learning, data mining, model training, feature selection. In classification, a model separates classes or predicts labels from data. The EA can search over model parameters, features, rules, trees, or neural-network hyperparameters.

3. Simulation problems (*probleme de simulare*)

Known: inputs and model. Must compute: outputs.

Examples: "what-if?" simulation, economic simulation, scenarios where the question is: "Given a model and input assumptions, what output will occur?"

MCQ traps

The tested answer set is often **optimization + modeling + simulation**. Distractors include "big-data processing," "validation of data," "dynamic memory allocation," or "autonomous vehicle movement." These may be domains where algorithms could be used indirectly, but in the course taxonomy they are not the three fundamental problem classes. "Search in the solution space" is a general process behind optimization, not always the named class in the official triad.

Supreme memory card

- Official problem classes = optimization, modeling/system identification, simulation.
- Classic model: inputs -> model -> outputs; class depends on what is known/unknown.
- Distractors: big data processing, validation, memory allocation, autonomous vehicle movement.

High-yield MCQ overlay from the improved edition

Exam Frequency: Very High — appears 10+ times, often as first or second question

ANSWER PATTERN: 1, 4, 6 — This is one of the most frequently tested facts.

Practice exercises and drills

Exercise 2.1 — Problem Classification: For each scenario, identify which of the three EC problem classes it belongs to (optimization, modeling, or simulation). If none, explain why:

- (a) Finding the best neural network architecture for image recognition
- (b) Predicting stock prices given historical data and a trained model
- (c) Determining the physical laws governing planetary motion from observed trajectories
- (d) Processing 10TB of customer data to find patterns
- (e) Simulating the economic impact of a new tax policy

Exercise 2.2 — Known vs Unknown: For each problem class, draw a table showing what is KNOWN and what must be COMPUTED. Then, for a real problem of your choice (e.g., finding the shortest path in a graph), classify it and identify what is known vs unknown.

Exercise 2.3 — Why Not Distractors: For each distractor (big-data, validation, memory allocation, autonomous vehicles), explain WHY it is not an EC problem class. What type of computational approach WOULD be appropriate for each?

Exercise 2.4 — Reverse Engineering: Given that EC must compute the "inputs" for a known model and desired output, design a specific optimization problem in economics. What are the inputs? What is the model? What is the desired output?

CHAPTER 3 — General EA structure and components

Final-edition quick exam handle

Exam frequency: Very high. Answer pattern: 1,3,5,7,10,12,14,15 = representation, fitness, population, parent selection, variation operators, survivor selection, initialization, termination.

A general evolutionary algorithm has a stable cycle:

1. Define the candidate/solution space.
2. Choose a representation and encoding/decoding.
3. Generate the initial population.
4. Evaluate candidates using the fitness/evaluation function.
5. Select parents from the current evaluated population.
6. Apply recombination to selected parents.
7. Apply mutation to descendants.
8. Evaluate new descendants.
9. Select the next generation/survivors.
10. Repeat until a termination condition is met.

Mandatory EA components are:

- **representation** (*reprezentarea*);
- **fitness/evaluation function** (*funcția de evaluare / fitness*);
- **population** (*populația*);
- **parent selection mechanism** (*mecanismul de selectare a părinților*);
- **variation operators** (*operatorii de variație*): recombination and mutation;
- **survivor or next-generation selection mechanism** (*selectarea supraviețuitorilor / generației următoare*);
- **initialization module** (*inițializarea populației inițiale*);
- **terminal condition** (*condiția terminală / de terminare*).

The exam frequently mixes true components with implementation accessories. Mutation probability and crossover probability are parameters, not core components in the official list. Random-number generation, graphical display of fitness evolution, generation of permutations, genetic-diversity calculation, and Hillclimbing are useful techniques or implementation details, but not core EA components in the standard answer.

Stochastic, deterministic and mixed stages

Initial population generation is usually stochastic. Parent selection is usually stochastic or probabilistic, although some deterministic choices can exist. Crossover and mutation are stochastic because the parents, gene positions, exchanged segments, and new alleles are usually chosen randomly. Survivor selection can be deterministic, stochastic, or mixed: elitist selection may deterministically preserve the best individual; tournament or probabilistic replacement may use random factors; age-based selection may ignore quality. The exam likes the distinction "always random" versus "sometimes random." The safe answer is: **many EA stages may use stochastic factors, but not every selection mechanism is always stochastic.**

Supreme memory card

- Mandatory components: representation, fitness, population, parent selection, variation operators, survivor selection, initialization, termination.
- Mutation/crossover probability are parameters, not mandatory components.
- General order: initialize -> evaluate -> parent selection -> recombination -> mutation -> evaluate descendants -> survivor selection.

High-yield MCQ overlay from the improved edition

Exam Frequency: Very High — appears 8+ sessions

ANSWER PATTERN: 1, 3, 5, 7, 10, 12, 14, 15 (in 15-item MCQ numbering)

NOT components (exam traps): probability of mutation (2), probability of recombination (4), random number generation (6), permutation generation (8), graphical quality representation (9), establishing genetic diversity (11), Hillclimbing (13).

Exam trap: "Selectia generatiei urmatoare" **can be deterministic** — TRUE and tested.

Practice exercises and drills

Exercise 3.1 — Component Identification: Given the following list of 15 items, select the 8 mandatory EA components. For each item you exclude, explain why it is NOT a mandatory component:

1. Reprezentarea, 2. Probabilitatea de mutatie, 3. Functia de evaluare, 4. Probabilitatea de recombinare, 5. Populatia, 6. Generarea de numere aleatoare, 7. Mecanismul de selectie a parintilor, 8. Generarea de permutari, 9. Reprezentarea grafica, 10. Operatorii de variatie, 11. Stabilirea diversitatii, 12. Mecanismul de inlocuire, 13. Hillclimbing, 14. Initializarea, 15. Conditia de terminare

Exercise 3.2 — Stochastic vs Deterministic: For each EA stage, determine whether it **MUST** be stochastic, **CAN** be deterministic, or **MUST** be deterministic. Give a concrete example for each. Why is survivor selection a stage that **CAN** be deterministic, even though many other stages are stochastic?

Exercise 3.3 — Algorithm Tracing: Trace the EA algorithm (steps 1-7) for a simple problem: maximize $f(x) = x^2$ where $x \in \{0,1,2,3,4,5\}$, population size 3, binary representation. Start with population {011, 101, 010} (values 3, 5, 2). Show one complete iteration.

Exercise 3.4 — Black Box Mapping: Map the black box model to the full EA algorithm. At which step does "Parent Pool" get created? At which step do "Descendants (recombination)" become "Descendants (mutation)"? What happens to the "Current Generation" after mutation?

CHAPTER 4 — Representation, genotype, phenotype, encoding and decoding

Final-edition quick exam handle

Exam frequency: High. Answer pattern: representation depends on the problem and influences solution quality; it also determines valid operators.

Representation (*reprezentarea*) establishes the connection between the real problem and the space in which the evolutionary algorithm searches. The real solution is the **phenotype** (*fenotip*). The algorithmic encoding of that solution is the **genotype/chromosome** (*genotip / cromozom*). A chromosome contains values called **alleles** (*alele*) placed at positions called **genes** (*gene*).

Encoding (*codificare*) maps phenotype -> genotype. **Decoding** (*decodificare*) maps genotype -> phenotype.

Representation includes both the encoding/decoding rule and the data structure used to define the genotype space. It is not chosen arbitrarily. It depends on the problem, influences the quality of the search, and determines which mutation and recombination operators are valid.

Exam-safe rules:

- Binary representation is natural for Boolean decision variables: select/not select, left/right, yes/no.
- Integer-string representation is preferred when each gene can take more than two distinct values, including ordinal or categorical discrete attributes.
- Real-valued vectors are used for continuous domains and numerical optimization.
- Permutations are used for ordering, sequencing, assignment, routing, scheduling and N-Queens style problems.
- Permutation representation requires special variation operators because ordinary crossover can create duplicates and omit required elements.
- Representation must generally allow decoding into a meaningful candidate solution. In constrained problems, either only feasible genotypes are generated or repair/penalty mechanisms are used.

A common MCQ wording is: "reprezentarea prin permutări nu permite mai mult de două gene cu aceeași valoare." The precise theoretical rule is stronger: in a valid permutation, **no element is repeated**. Some MCQ keys mark only "needs special operators," while others also mark "does not allow repeated values." Treat both as theoretically true, but see the exam-risk notes below.

Supreme memory card

- Representation bridges phenotype/problem solution and genotype/algorithm chromosome.
- Gene = position; allele = value; chromosome/genotype = encoded candidate.
- Representation depends on the problem, influences solution quality, and determines valid operators.

High-yield MCQ overlay from the improved edition

Exam Frequency: High — appears 6+ sessions

Exam rule: Decoding is **mandatory after extracting the best individual from the final population**. It depends on the representation and does NOT guarantee optimal results.

ANSWER PATTERN: 2, 5 — representation depends on problem AND influences solution quality.

Practice exercises and drills

Exercise 4.1 — Representation Design: For each problem below, design an appropriate genotype representation. Identify the gene, the allele, and the allele domain for each:

- (a) Selecting a subset of 20 products to include in a promotion
- (b) Assigning 10 employees to 10 different shifts
- (c) Finding the optimal temperature setting (continuous, 18-30°C) for a chemical reactor
- (d) Determining the order in which a delivery truck visits 15 locations

Exercise 4.2 — Encoding/Decoding Practice: Given a binary representation for the 0-1 knapsack problem with 5 items, encode and decode:

- Phenotype: select items {2, 4, 5} → what is the genotype?
- Genotype: [1, 0, 1, 1, 0] → what is the phenotype?

Why is decoding necessary even when the genotype seems "obvious"?

Exercise 4.3 — Representation Impact: The same TSP problem (5 cities) can be represented as a permutation OR as a binary string of length 5×5 (adjacency matrix). Compare these two representations in terms of: search space size, valid operators, constraint handling, and likelihood of producing feasible solutions.

Exercise 4.4 — True/False with Reasoning: For each statement about representation, mark TRUE or FALSE and explain:

- (a) The representation choice can be made randomly without affecting results.
- (b) The representation determines which variation operators are valid.
- (c) The representation influences the quality of solutions found.
- (d) Decoding is optional if the genotype directly represents the solution.

CHAPTER 5 — Main GA representations

Final-edition quick exam handle

Exam frequency: High. Binary = first/canonical; integer = more than two values per gene; real = continuous; permutation = ordering/no duplicates/special operators.

A. Binary strings

Binary strings were the first classical GA chromosome representation and remain the canonical representation. A chromosome is a sequence of 0/1 genes. It is appropriate when variables are Boolean or when numerical variables are encoded into binary or Gray code.

Typical operators:

- mutation: bit flip / negation (*mutație prin negare*), changing 0 to 1 or 1 to 0;
- recombination: one-point, multi-point, uniform crossover.

Example: chromosome 1 0 1 1 0; mutate genes 2 and 5 -> 1 1 1 1 1.

MCQ traps: binary representation is used in GA and was classical/canonical, but it does not always lead to optimal results, and it is not independent of the problem.

B. Integer strings

Integer representation uses genes taking values from integer sets. It is preferred when a gene has more than two possible values: machine type, task category, ordinal score, quantity, discrete coordinate, number of units purchased, etc.

Typical operators:

- mutation by random reset: replace a gene with another valid integer;
- creep mutation (*mutație prin fluaj*): increase or decrease an integer by a small amount;
- one-point, multi-point and uniform crossover can often be applied position-wise;
- arithmetic recombination may be used if the result is rounded and remains valid.

Example: [4, 2, 7, 1]; creep at gene 3 by +1 -> [4, 2, 8, 1].

MCQ trap: arithmetic recombination for integer strings is not "forbidden"; it can be applied with rounding. If an answer says it is valid only when all pairs of alleles have the same parity, that is not the theoretical rule and is likely a corrupted key.

C. Real-valued vectors

Real-valued chromosomes represent continuous variables: $\mathbf{x} = (x_1, x_2, \dots, x_n)$. They are used in continuous optimization and ES-like problems.

Typical mutation:

- uniform mutation: replace a real value with a random value from its allowed interval;
- non-uniform mutation: change magnitude depends on generation or distribution;
- Gaussian perturbation: $x_i' = x_i + N(0, \sigma)$.

Typical recombination:

- simple arithmetic recombination: one part copied, one gene or segment computed as a weighted average;
- single/singular arithmetic recombination: one selected gene is arithmetically recombined;
- total/global arithmetic recombination: all genes are combined by weighted averages.

Example: parents have gene values 5.8 and 3.5; with $\alpha = 0.3$, child values can be $0.3 \cdot 5.8 + 0.7 \cdot 3.5 = 4.19$ and $0.7 \cdot 5.8 + 0.3 \cdot 3.5 = 5.11$.

D. Permutations

A permutation chromosome is an ordering of distinct elements, e.g. [2, 7, 5, 8, 1, 4, 6, 3]. It is used when order or assignment is essential.

Applications:

- TSP route order;
- N-Queens, where each position encodes a row/column placement;
- scheduling/activity sequencing;
- transportation allocation order;
- puzzle/placement problems.

Typical mutation:

- swap/interchange (*interscimbare*);
- insertion (*inserare*);
- inversion (*inversiune*);
- scramble (*amestec*).

Typical recombination:

- PMX;
- OX/OCX;
- CX;
- ECX.

MCQ trap: ordinary one-point crossover can produce duplicate alleles in permutation problems; special operators preserve permutation validity.

Supreme memory card

- Binary: first classical GA representation; bit-flip mutation; one/multi/uniform crossover.
- Integer: preferred when each gene has more than two possible values; creep/fluaj and random reset.
- Real: continuous variables; uniform/non-uniform/Gaussian mutation; arithmetic recombination.
- Permutation: no duplicates; requires PMX/OX/CX/ECX and special mutation.

High-yield MCQ overlay from the improved edition

Exam facts: First representation. Most used. Natural for boolean/logical variables.

ANSWER PATTERN: Preferred when each gene has > 2 distinct values.

Valid operators for real-valued (MCQ pattern: 5,6,14,15,16,18,19,20): uniform mutation, non-uniform mutation, one-point, multi-point, uniform recombination, simple/singular/total arithmetic recombination, local mutation.

Key answer: Requires special variation operators. No duplicates.

Practice exercises and drills

Exercise 5.1 — Gray Code: Convert the number 7 to both standard binary and Gray code (4 bits). Then convert 8. How many bits differ between the standard binary representations of 7 and 8? How many bits differ between their Gray code representations? Why does this matter for mutation behavior?

Exercise 5.2 — Representation Selection: For each problem, choose the most appropriate representation and justify:

- (a) Finding optimal weights in a neural network
- (b) Assigning 50 students to 50 dorm rooms
- (c) Deciding which of 100 investments to include in a portfolio
- (d) Determining the number of units (0-100) to order for each of 20 products

Exercise 5.3 — Operator Compatibility: Given an integer string representation where each gene can be 0-10, which crossover operators are valid? Can arithmetic crossover be applied? If so, what additional step is needed? Show a worked example: Parent1 = [3, 7, 2], Parent2 = [8, 1, 9], $\alpha=0.4$, cut at position 2.

Exercise 5.4 — Permutation Properties: Given $P = [3, 1, 4, 2, 5]$, apply each permutation mutation operator and show the result:

- (a) Swap positions 2 and 4
- (b) Inversion between positions 1 and 3
- (c) Scramble between positions 2 and 4 (give one valid result)
- (d) Insertion: remove element at position 4, insert at position 1

Exercise 5.5 — Space Size Calculation: Calculate the search space size for: (a) 10-bit binary, (b) integer strings of length 5 with values 0-9, (c) permutations of 8 elements, (d) real-valued vectors of length 3 (continuous). Which is largest? Which is most constrained?

CHAPTER 6 — Fitness/evaluation function

Final-edition quick exam handle

Exam frequency: Very high. Main pattern: fitness evaluates each candidate/individual. Quality-function variants may expect: evaluates candidate + is maximized.

The fitness/evaluation function (*funcția fitness / funcția de evaluare / funcția de calitate*) assigns a quality value to each candidate/individual. It evaluates the candidate, not the algorithm. It is the basis for parent selection and survivor selection, but it does not "select by itself." Selection mechanisms use the fitness values.

The course emphasizes that if an objective must be minimized, the algorithm may transform it into a maximization problem by maximizing $-f$ or, where valid, $1/f$. For example, TSP minimizes route cost; a GA may maximize $\text{fitness}(\text{route}) = 1 / \text{cost}(\text{route})$, provided cost is positive. In constrained problems, the fitness may include penalties or feasibility checks, but the fundamental role remains: measure candidate quality.

MCQ traps:

- Fitness does **not** evaluate the speed of the algorithm.
- Fitness does **not** evaluate resource consumption of the search.

- Fitness does **not** have to be randomly selected.
- Fitness is not changed at every iteration merely because evolution occurs.
- Fitness does not directly choose parents or survivors; selection mechanisms choose using fitness.
- Fitness evaluates each candidate, not the current population compared with the initial or previous population.

Use this formula for FPS when all fitness values are positive:

$$p_i = f_i / \sum(f_j)$$

where p_i is the selection probability of individual i .

Supreme memory card

- Fitness evaluates each candidate/individual, not the algorithm, speed, or resources.
- Selection mechanisms use fitness; fitness does not select by itself.
- Minimization can be transformed by $-f$ or by $1/f$ only when valid and safe.

High-yield MCQ overlay from the improved edition

Exam Frequency: Very High — appears 6+ sessions

ANSWER PATTERN: Option 9 or "i" — evaluates each candidate. (In some versions: 2, 4 — evaluates candidate + must be maximized.)

Exam-risk note: MCQ asks about "functia de tip calitate" → answer is **2, 4** (evaluates candidate + must be maximized).

Practice exercises and drills

Exercise 6.1 — Fitness Design: For each problem, design a fitness function. State whether it should be maximized or minimized, and if minimized, show how to transform it:

- (a) Minimize total travel distance in TSP
- (b) Maximize number of non-attacking queen pairs in N-Queens
- (c) Minimize energy consumption in a sensor network deployment

Exercise 6.2 — What Fitness Evaluates: For each statement, determine if it describes what the fitness function DOES or DOES NOT do. Explain your reasoning:

- (a) "The fitness function determines which individual is selected for reproduction."
- (b) "The fitness function measures how well candidate solution X solves problem Y."
- (c) "The fitness function changes its definition when the population converges."
- (d) "The fitness function is used as the basis for selection mechanisms."

Exercise 6.3 — Fitness Transformation: Given a minimization problem where $f(x)$ can take negative values (e.g., $f(x) \in [-100, 50]$), can you use $1/f(x)$ to transform it to maximization? Why or why not? What transformation WOULD work?

Exercise 6.4 — Fitness Landscape: Sketch (describe in words) the fitness landscape for a simple problem: $f(x) = \sin(x)$ for $x \in [0, 4\pi]$. How many local optima? How many global optima? Why is this a multimodal problem? How would an EA navigate this landscape?

CHAPTER 7 — Population, diversity, initialization and termination

Final-edition quick exam handle

Exam frequency: High. Initial population: before evolution, random, often/course-key uniform, once. Termination: mandatory; max iterations/generations is the safest criterion.

A population (*populație*) is a multiset of genotypes. "Multiset" matters: duplicates may occur unless the algorithm explicitly prevents them. The individual is static: it does not evolve or adapt internally. The population is dynamic: it evolves by replacement across generations.

Population types:

- fixed size: simplest and most common;
- variable size: possible but less central.

Population models often tested:

- generational model (*model generațional*): a whole generation or a large part is replaced;
- steady-state / stable-state model (*stare stabilă / stable state*): only a small number of individuals are replaced at a time;
- elitist/superelitist variants: best old individual(s) are preserved.

Diversity (*diversitate genetică*) can be measured by:

- number of distinct genotypes/phenotypes;
- number of distinct fitness values;
- entropy-like measures.

The initial population (*populația inițială*) is created before evolution proper, usually randomly, often uniformly. It is not regenerated at the start of every evolutionary cycle. In constrained problems, algorithms may generate only feasible initial individuals, or generate then repair/filter candidates.

Termination condition (*condiția de terminare*) is mandatory. Valid criteria include:

- maximum number of generations/iterations;
- maximum number of fitness evaluations;
- target or optimal fitness reached;
- no significant improvement for a specified number of generations;
- diversity below a threshold;
- computational limit.

MCQ traps: "available memory" may be a computational constraint in general, but the course's explicit termination choices usually focus on generations, objective threshold/improvement and diversity. "Population size decreases below a limit" and "chromosome length increases" are not standard termination conditions in the tested list.

Supreme memory card

- Population = multiset of genotypes; duplicates may exist unless forbidden by representation/rules.
- Initial population is generated before evolution, usually randomly, often uniformly; not every iteration.
- Termination can use max generations/evaluations, target quality, no improvement, diversity, computational limit.

High-yield MCQ overlay from the improved edition

Exam Frequency: High

ANSWER PATTERN: Generated before evolution (c), randomly (f), using uniform distribution (e). Built once.

ANSWER PATTERN: Mandatory (a) + includes iteration control (c). Valid: **a, c** or **a, b, c** depending on version.

INVALID: memory availability, decreasing population size, increasing chromosome length

Practice exercises and drills

Exercise 7.1 — Initialization Design: For the 0-1 knapsack problem with $C_{max} = 50$ and items with costs $[10, 15, 8, 20, 12, 5]$, design an initialization procedure that generates only feasible individuals. Show how you would generate 5 individuals randomly while ensuring $\sum(cost_i \times x_i) \leq 50$.

Exercise 7.2 — Diversity Measurement: Given a population of 10 individuals with fitness values $[45, 52, 45, 60, 52, 70, 45, 60, 52, 70]$, calculate: (a) number of distinct fitness values, (b) number of distinct genotypes if you know the mapping, (c) why is diversity important? What happens when diversity = 1?

Exercise 7.3 — Termination Design: Design a termination condition that combines at least two valid criteria. For example: "Stop when either (a) 100 generations have passed OR (b) the best fitness hasn't improved by more than 0.01 in the last 20 generations." Explain why each criterion is needed and what could go wrong with only one.

Exercise 7.4 — Population State Tracking: Trace the population through 2 generations of a GA. Start with $P_0 = \{A, B, C, D\}$ (fitness: 10, 20, 15, 25). After parent selection, mating pool = $\{D, B, B, D\}$. After crossover and mutation, offspring = $\{E, F, G, H\}$ (fitness: 30, 12, 22, 18). After survivor selection (elitist, keep best 4), $P_1 = ?$ Show each population state.

CHAPTER 8 — Direct search, stochastic search and Hillclimbing

Final-edition quick exam handle

Exam frequency: High. Hillclimbing pattern: one point, can be restarted, sometimes global, usually local optimum; not temperature-based.

Hillclimbing is an iterative local search technique, positioned in the course as part of the evolution from direct search to stochastic search. It operates on a single current point/candidate. The algorithm chooses an initial point, examines neighbors, moves to a better neighbor, and stops when no neighbor improves the current solution.

Basic Hillclimbing:

1. Choose an initial candidate, often randomly.
2. Generate neighbors of the current candidate.
3. Select a better neighbor, often the best available.
4. Replace the current candidate with that neighbor.
5. Stop when no better neighbor exists.

Hillclimbing usually finds a **local optimum**, not necessarily a global optimum. In multimodal landscapes, the starting point strongly affects the result. Repeated or multi-start Hillclimbing improves performance: run from many initial points and keep the best result.

Neighbor generation depends on representation:

- binary: flip one bit;
- integer/real: add or subtract a step;
- permutation: swap/transposition, insertion, inversion.

Exam traps:

- Hillclimbing does not always find the global optimum.
 - It is not simulated annealing; temperature reaching zero is a simulated-annealing idea, not a Hillclimbing termination rule.
 - It is not restricted only to real-valued strings.
 - In the MCQ corpus, some keys label Hillclimbing inconsistently. The safe theory is: Hillclimbing is a local search method, usually not a population EA by itself, but can be embedded in memetic algorithms.
-

Supreme memory card

- Hillclimbing uses one current point and neighbors; usually finds local optimum, not guaranteed global optimum.
- Multi-start improves chances but does not guarantee optimum.
- Memetic algorithms combine EA global search with local search such as hillclimbing.

High-yield MCQ overlay from the improved edition

Exam Frequency: High — appears 5+ sessions

ANSWER PATTERN: 1, 2, 5, 7

Practice exercises and drills

Exercise 8.1 — Hillclimbing Trace: Apply Hillclimbing (single start) to maximize $f(x) = -x^2 + 10x$ for $x \in \{0, 1, \dots, 10\}$. Start at $x=2$. Use neighbors defined as $x \pm 1$ (clipped to domain). Show each iteration: current point, neighbors, best neighbor, decision (move or stop). What is the final result? Is it the global optimum?

Exercise 8.2 — Multi-Start Analysis: Using the same function $f(x) = -x^2 + 10x$, run Hillclimbing from starting points $x=0$, $x=3$, $x=7$, $x=10$. For each, show the trajectory. How many different final points are found? Which starting point leads to the global optimum? Why?

Exercise 8.3 — Neighbor Design: For the N-Queens problem with permutation representation $P = [1, 2, 3, 4]$, list ALL neighbors using the transposition operator. How many neighbors are there? For each neighbor, calculate the fitness (number of non-attacking pairs). Which neighbor would Hillclimbing choose?

Exercise 8.4 — Conceptual: Explain why Hillclimbing is described as "a bridge between deterministic and evolutionary search" in the course. What deterministic aspects does it have? What stochastic aspects? Why is it considered an evolutionary algorithm in this course despite having no population?

CHAPTER 9 — Mutation operators

Final-edition quick exam handle

Exam frequency: Medium-high. GA mutation = low probability, usually on descendants after recombination; permutation mutations = swap, insertion, inversion, scramble/amestec.

Mutation introduces random changes into individuals and helps maintain diversity, explore new regions, and escape local optima. In classical GA, mutation has low probability. It is usually applied to descendants after recombination and before survivor selection. Mutation may produce infeasible individuals in constrained problems; therefore repair, rejection, penalties or special feasible operators may be necessary.

A. Binary mutation: bit flip / negation

Change selected bits: 0 → 1, 1 → 0. Example: 1 1 0 1 0; mutate positions 2 and 5 → 1 0 0 1 1.

Trap: mutation is not applied only when diversity falls below a threshold; that can be an adaptive policy, but classical GA uses mutation probabilistically.

B. Integer mutation: creep/fluaj and random replacement

Creep mutation changes a gene by a small integer step. Example: [6, 1, 8, 10]; creep gene 3 by +1 → [6, 1, 9, 10].

Random replacement substitutes a gene with another valid allele. Example: [2, 4, 1, 3]; gene 2 reset to 7 → [2, 7, 1, 3].

Trap: for a creep mutation, most genes remain identical; only one or a few numeric values change slightly.

C. Real mutation: uniform, non-uniform, Gaussian

Uniform mutation replaces a real value with a random value from its allowed interval. Non-uniform mutation changes with a distribution that may depend on the generation. Gaussian mutation adds noise: $x_i' = x_i + N(0, \sigma)$.

Example: [0.4, 0.1, 0.7]; Gaussian mutation at gene 2 with noise +0.06 → [0.4, 0.16, 0.7].

D. Permutation mutation

Permutation mutation must preserve the "each element exactly once" property.

Swap/interchange (interscimbare). Pick two positions and swap values. [1, 2, 3, 4, 5], swap positions 2 and 5 → [1, 5, 3, 4, 2].

Insertion (inserare). Remove one element and insert it elsewhere. [1, 2, 3, 4, 5], move value at position 2 after position 4 → [1, 3, 4, 2, 5].

Inversion (inversiune). Reverse a segment. [1, 2, 3, 4, 5, 6], invert positions 2..5 → [1, 5, 4, 3, 2, 6].

Scramble/amestec. Randomly shuffle the values inside a chosen segment, leaving outside positions fixed.

[1, 2, 3, 4, 5, 6, 7], scramble positions 2..6 → [1, 5, 3, 6, 2, 4, 7].

MCQ pattern: compare original and mutated permutation. Positions outside the mutation interval remain identical; inside the interval, the same elements appear in a different order. For scramble, the two cut positions are the smallest and largest positions where original and mutated chromosomes differ.

Supreme memory card

- Mutation maintains diversity, enables exploration, and may escape local optima.
- Classical GA: low mutation probability, generally after recombination, applied to descendants.
- Permutation mutation types: swap, insertion, inversion, scramble/amestec.

Practice exercises and drills

Exercise 9.1 — Creep Identification: Given $X = [6, 1, 8, 10, 5, 7, 12, 43, 9, 3, 4, 2]$, determine which of these are valid creep mutation results and explain why: (a) [6, 1, 8, 10, 5, 7, 12, 43, 9, 4, 3, 2] (b) [7, 1, 8, 10, 5, 7, 12, 43, 9, 3, 4, 2] (c) [6, 1, 8, 10, 5, 7, 12, 43, 9, 3, 4, 2] (d) [16, 1, 8, 10, 5, 7, 12, 43, 9, 3, 4, 2] (e) [6, 1, 8, 10, 15, 7, 12, 43, 9, 3, 4, 2] (f) [6, 1, 8, 10, 5, 7, 12, **42**, 9, 3, 4, 2]

Exercise 9.2 — Permutation Mutation Application: Given $P = [4, 3, 6, 5, 7, 2, 1, 8]$, apply each mutation operator for positions (2,5) (1-indexed). Show the exact result: (a) Inversion: reverse segment from position 2 to 5 (b) Swap: swap values at positions 2 and 5 (c) Insertion: remove element at position 5, insert at position 2 (d) Scramble: give one valid random shuffle of segment positions 2-5

Exercise 9.3 — Mutation Probability: In a GA with population size 100 and chromosome length 20, if mutation probability $p_m = 1/(\text{dim} \times n) = 1/2000$, approximately how many genes will be mutated per generation on average? What if $p_m = 1/n = 1/20$? Explain why constrained problems need p_m closer to $1/n$.

Exercise 9.4 — Feasibility: A knapsack solution $[1, 0, 1, 0, 1]$ with costs $[10, 8, 12, 6, 5]$ and $C_{\max} = 25$ is feasible (cost=27... wait, $10+12+5=27 > 25$, NOT feasible). If mutation flips position 3 (value 1→0), is the result feasible? Show the calculation. Why does the course say "if result is not feasible, do NOT retry mutation"?

CHAPTER 10 — Recombination/crossover operators

Final-edition quick exam handle

Exam frequency: High. Recombination pattern: after parent selection, before mutation, high probability in classical GA, iterative, uses parent pool; usually two parents.

Recombination (*recombine* / *crossover*) combines genetic material from parents. In GA it is usually binary: two parents produce one or two descendants. It usually occurs with high probability in the classical GA and is applied after parent selection and before mutation. It uses the parent population/mating pool. It is performed iteratively during evolution, not only once. It is not restricted to unconstrained problems, although constraints may require repair or special operators.

A. One-point crossover

Choose one cut point. Child 1 receives the prefix of parent 1 and suffix of parent 2; child 2 receives prefix of parent 2 and suffix of parent 1.

Example: $P1 = 111 | 000$, $P2 = 000 | 111 \rightarrow C1 = 111111$, $C2 = 000000$.

B. Multi-point crossover

Choose several cut points and alternate copied segments.

Example with cuts after positions 2 and 5: $P1 = 11 | 000 | 11$, $P2 = 00 | 111 | 00 \rightarrow C1 = 11 \ 111 \ 11$, $C2 = 00 \ 000 \ 00$.

C. Uniform crossover

For each gene independently, choose from parent 1 or parent 2, often with probability 0.5. It is highly disruptive but flexible.

D. Arithmetic recombination

For numeric chromosomes, compute weighted averages.

For real genes a and b , with α in $[0, 1]$:

$$\text{child1} = \alpha \cdot a + (1 - \alpha) \cdot b \quad \text{child2} = (1 - \alpha) \cdot a + \alpha \cdot b$$

For integers, round the result if permitted and ensure values remain valid. For permutations, arithmetic recombination is generally invalid because averages of labels do not preserve the permutation property.

What to do if recombination creates aberrant descendants

If a recombination operator creates invalid or aberrant individuals, common strategies are:

- use parents unchanged as descendants;
- repair descendants;
- reject and regenerate;
- let selection eliminate weak/invalid individuals if they are evaluable with penalties.

Some MCQs state that the algorithm can still produce correct results through selection, but this is not a license to ignore invalid encoding.

Supreme memory card

- Recombination usually uses two parents, high probability in canonical GA, before mutation.
- One-point/multi-point/uniform operate positionally for strings/vectors.
- Arithmetic recombination uses weighted averages; integer version needs rounding and feasibility checks.

High-yield MCQ overlay from the improved edition

Exam Frequency: High

ANSWER PATTERN (recombination): e,f,g,h,j — applied after selection, before mutation, uses parent population, high probability, iterative.

Exam fact: Better results only in **particular cases** — not always better.

Exam answer: Use parents as descendants (se folosesc parintii ca descendentii).

Practice exercises and drills

Exercise 10.1 — Crossover Tracing: Given $P1 = [1,0,1,1,0,1,0,0]$ and $P2 = [0,1,0,1,1,0,1,1]$, produce offspring using: (a) One-point crossover at position 4 (b) Two-point crossover at positions 2 and 6 (c) Uniform crossover with coin flips: H,T,H,H,T,T,H,T ($H = P1$, $T = P2$)

Exercise 10.2 — Arithmetic Crossover: Given $P1 = [10, 20, 30, 40]$ and $P2 = [20, 10, 40, 20]$, $\alpha = 0.3$: (a) Simple arithmetic at position 3: compute both children (b) Total arithmetic: compute both children (c) If these were integers, what rounding would you apply? (d) Why does the course say total arithmetic is "incorrect" for the knapsack problem?

Exercise 10.3 — Asexual Recombination: In a GA with $p_r = 0.8$ and population of 4, how many crossover operations are expected? What happens to parents not selected for crossover? Why is this called "asexual recombination" and not "no recombination"?

Exercise 10.4 — Constraint Handling: A constrained problem requires all solutions to satisfy $\sum x_i \leq 100$. $P1 = [30, 40, 20, 10]$ and $P2 = [20, 30, 50, 15]$ are both feasible. One-point crossover at position 3 produces: $C1 = [30, 40, 50, 15]$ and $C2 = [20, 30, 20, 10]$. Check feasibility of each offspring. What should be done with infeasible offspring according to the course?

CHAPTER 11 — Permutation crossover operators: PMX, OCX/OX, CX, ECX

Final-edition quick exam handle

Exam frequency: Very high. Operator matching: PMX = mapping/segment repair; OX/OCX = relative order; CX = cycles/positions; ECX = edges/adjacency.

Permutation crossover is critical for the exam because many calculation-style MCQs use it.

A. PMX — Partially Matched / Partially Mapped Crossover

PMX aims to preserve position information and mapping relations while maintaining a valid permutation. The course associates PMX with adjacency-dependent problems in some slides; in broader GA terminology, PMX is a standard permutation operator that preserves partial mappings and absolute positions in the cut segment.

Method:

1. Choose two cut points.
2. Copy the segment from parent 1 into child 1.
3. Build mappings between the two parent segments.
4. For positions outside the segment, copy values from parent 2 if they do not conflict.
5. If a copied value conflicts with the segment, follow the mapping chain until a non-conflicting value is found.
6. Fill all remaining positions.
7. Produce the symmetric child by reversing parent roles.

Example: P1 = [1, 2, 3, 4, 5, 6, 7, 8] P2 = [3, 7, 5, 1, 6, 8, 2, 4] Cut positions 3..6: P1 segment [3, 4, 5, 6], P2 segment [5, 1, 6, 8].

Child 1 starts: [_, _, 3, 4, 5, 6, _, _]. Using PMX repair and filling from P2 gives: [8, 7, 3, 4, 5, 6, 2, 1]. Child 2 symmetrically gives: [4, 2, 5, 1, 6, 8, 7, 3].

Exam method: always verify the final child contains every allele exactly once.

B. OCX/OX — Order Crossover

OX/OCX preserves relative order. It is used when the permutation indicates the order of events (*ordinea de apariție a evenimentelor*).

Method:

1. Choose two cut points.
2. Copy the segment from parent 1 into the child.
3. Starting after the second cut in parent 2, scan cyclically.
4. Insert values not already present into the child, preserving their scanned order.

Example: P1 = [1, 2, 3, 4, 5, 6, 7, 8] P2 = [3, 7, 5, 1, 6, 8, 2, 4] Cut 3..6 copies [3, 4, 5, 6]. Scanning P2 after position 6 gives 2, 4, 3, 7, 5, 1, 6, 8; omit existing 3,4,5,6; insert 2, 7, 1, 8. Child: [1, 8, 3, 4, 5, 6, 2, 7] depending on insertion convention; what matters is that non-segment elements preserve their order from the other parent.

C. CX — Cycle Crossover

CX preserves absolute allele positions by identifying cycles between parents. A cycle is formed by looking at a position, taking the value from parent 2 at that position, finding where that value occurs in parent 1, and repeating until returning to the starting position.

Method:

1. Start at an unvisited position.
2. Build a cycle using value-position mapping between the two parents.
3. Copy one cycle from parent 1 into child 1, the same cycle from parent 2 into child 2.
4. Alternate cycles between parents for remaining cycles.

Example: P1 = [1, 2, 3, 4, 5, 6, 7, 8] P2 = [3, 7, 5, 1, 6, 8, 2, 4]

Cycles by position:

- Cycle 1: positions 1,3,5,6,8,4

- Cycle 2: positions 2,7

Alternate cycles: Child 1 = [1, 7, 3, 4, 5, 6, 2, 8] Child 2 = [3, 2, 5, 1, 6, 8, 7, 4]

Exam-risk note: in the uploaded MCQ corpus, one CX question involving $X = [6, 1, 8, 10, 5, 7, 9, 3, 4, 2]$ and $Y = [9, 8, 7, 3, 6, 1, 5, 10, 4, 2]$ has inconsistent answer keys. Standard cycle analysis gives cycles (1, 7, 5), (2, 3, 6), (4, 8), (9), (10) in 1-based positions, and the standard alternating child corresponds to the option with $C1 = [6, 8, 7, 10, 5, 1, 9, 3, 4, 2]$, not the corrupted variant that swaps 7 and 8. If the exam key follows the corrupted file, this is a risk; theoretically, use the cycle method.

D. ECX — Edge Crossover

ECX preserves adjacency/edge relations. It creates offspring by preserving pairs of consecutive alleles present in at least one parent. It is especially suitable for TSP-like problems where route edges matter more than absolute positions.

Method:

1. Build an edge table: for each allele, list its adjacent neighbors in each parent.
2. Start from an allele.
3. Choose the next allele from its edge list, often preferring common edges or the neighbor with the smallest remaining edge list.
4. Remove used edges and continue until a valid permutation is built.

Comparison table

Operator	Preserves	Best for	Common exam trap
PMX	partial mapping and segment positions	permutation problems, often adjacency-dependent in course examples	treating it like one-point crossover
OX/OCX	relative order of elements	order of events, scheduling/order problems	expecting absolute positions to be preserved
CX	absolute positions through cycles	when allele-position inheritance matters	failing to compute cycles
ECX	adjacency/edge relations	TSP and route-like problems	confusing edges with positions

Supreme memory card

- PMX preserves a copied segment and repairs conflicts by mapping.
- OX/OCX preserves relative order.
- CX preserves value-position cycles.
- ECX preserves adjacency/edges, especially TSP-like problems.

High-yield MCQ overlay from the improved edition

Exam Frequency: Very High — calculation questions appear every session

Exam rules:

Practice exercises and drills

Exercise 11.1 — PMX Full Calculation: $P1 = [6, 3, 11, 7, 14, 8, 5, 15, 1, 2, 4, 13, 9, 10, 12]$, $P2 = [7, 1, 15, 13, 2, 14, 6, 10, 12, 11, 4, 8, 3, 9, 5]$, cuts at 4 and 8. Work through the complete PMX calculation: (a) What segment is copied from P1 to C1? (b) What is the

complete mapping? (c) Fill in C1 position by position, showing conflict resolution. (d) Verify: is the result {13,1,10,7,14,8,5,15,12,11,4,2,3,9,6}?

Exercise 11.2 — CX Cycle Construction: $X = [6,1,8,10,5,7,9,3,4,2]$, $Y = [9,8,7,3,6,1,5,10,4,2]$. (a) Find Cycle 1: start at position 1, trace through both parents. (b) Find Cycle 2: start at the first uncovered position. (c) Find remaining cycles. (d) Build Child 1: Cycle 1+3 from X, others from Y. (e) Verify: is the result [6,8,7,10,5,1,9,3,4,2]?

Exercise 11.3 — OCX with Wrap-Around: $P1 = [1,2,3,4,5,6,7,8,9]$, $P2 = [9,3,7,8,2,6,5,1,4]$, cuts at 3 and 7. (a) Copy $P1[3..7] = [3,4,5,6,7]$ to child. (b) Which values from P2 remain (skipping used)? (c) How do you fill positions 8,9, then wrap to 1,2? (d) What is the final child?

Exercise 11.4 — Operator Selection: For each problem description, choose the most appropriate crossover operator and justify:

- (a) "We need to preserve the exact positions of certain values in the permutation."
- (b) "The relative order in which events occur is critical."
- (c) "We want to preserve edges (adjacent pairs) from both parents."
- (d) "We have adjacency dependencies between elements."

CHAPTER 12 — Parent selection mechanisms

Final-edition quick exam handle

Exam frequency: High. Parent selection: current evaluated population; generally stochastic/nondeterministic; FPS, SUS, tournament, rank are key.

Parent selection (*selecția părinților*) chooses individuals from the current evaluated population to form the mating pool. It is usually nondeterministic or stochastic, but deterministic versions can exist. It is performed iteratively: for a population of size k , the selection process is repeated until enough parents are chosen. Parent selection does not depend primarily on chromosome representation; it depends on fitness and the selection mechanism.

Selection pressure is the degree to which better individuals are favored. Too much pressure causes premature convergence; too little pressure makes selection almost random.

A. FPS / fitness-proportionate selection / roulette wheel

For positive fitness values:

$$p_i = f_i / \sum(f_j)$$

The better the fitness, the larger the probability. Expected number of copies is roughly $f_i / \text{average}_f$.

Drawbacks:

- premature convergence when a few individuals are much better;
- weak pressure when fitness values are similar;
- sensitivity to scaling and translation of fitness values.

B. Windowing and sigma scaling

FPS variants adjust fitness values before computing probabilities. Sigma scaling reduces the dominance of extreme individuals and maintains useful selection pressure. "FPS cu alfa scalare" appears as a distractor/corrupted phrase in the MCQs; valid named variants in the course include windowing and sigma scaling.

C. Rank-based selection

Individuals are sorted by fitness, then probabilities are assigned by rank rather than raw fitness. This avoids raw-fitness scaling problems. Linear and nonlinear rank selection can appear.

D. SUS — Stochastic Universal Sampling

SUS is the multiple-arm roulette method (*ruletă multi-brăț*). Instead of spinning one pointer repeatedly, it uses equally spaced pointers in one spin. It produces selected samples closer to the intended probability distribution than simple roulette.

MCQ rule:

- simple roulette: rotate/spin k times to select k individuals;
- SUS / multi-arm roulette: one spin with k arms/pointers can complete the selection.

E. Tournament selection (*selecție prin turnir*)

Choose a small subset of individuals, then select the best or select probabilistically among them. Tournament selection is not always deterministic because the tournament participants are usually chosen randomly, and some variants select within the tournament probabilistically.

MCQ trap: "tournament selection is always deterministic" is false.

Supreme memory card

- Parent selection chooses reproduction candidates from current evaluated population.
- FPS/roulette proportional to fitness; rank selection uses rank; tournament compares sampled candidates; SUS uses equally spaced pointers.
- Parent selection is generally nondeterministic/stochastic, but deterministic variants may exist.

High-yield MCQ overlay from the improved edition

■ **Exam Frequency:** High

- **Formula:** $p_i = f_i / \sum f_j$

■ **Exam fact:** Produces results **closest to the calculated probability distribution**.

■ **Exam fact:** Does NOT respect probability distribution (nu respecta distributia de probabilitate de selectie).

Practice exercises and drills

Exercise 12.1 — FPS Calculation: Given population with fitness values [10, 30, 15, 40, 5]: (a) Calculate selection probability for each individual. (b) Calculate cumulated probabilities. (c) For $r = 0.25$, which individual is selected? (d) For $r = 0.72$, which individual is selected? (e) If you spin 5 times, approximately how many times would individual 4 be selected?

Exercise 12.2 — SUS Calculation: Same population [10, 30, 15, 40, 5], $\text{dim}=5$: (a) Calculate p_i and q_i . (b) If $r = 0.05$, the arm positions are: 0.05, 0.25, 0.45, 0.65, 0.85. Which individuals are selected? (c) Compare the selection counts with FPS — is SUS closer to the expected distribution?

Exercise 12.3 — Sigma Scaling: Given population fitness [10, 30, 15, 40, 5], calculate: (a) Mean $\bar{f}$ and standard deviation σ . (b) Adjusted fitness $g(y)$ with $c=2$ for each individual. (c) How does sigma scaling change the selection probabilities compared to raw FPS?

Exercise 12.4 — Tournament Analysis: For population [10, 30, 15, 40, 5] with $k=2$ tournament: (a) If tournament picks individuals 1 and 5, who wins? (b) Can the weakest individual (fitness 5) ever win a tournament? (c) Does tournament selection follow the FPS probability distribution? Explain why or why not.

CHAPTER 13 — Survivor selection / next-generation selection

Final-edition quick exam handle

Exam frequency: High. Survivor selection: next generation after descendants exist; may use current population, offspring, or both; may be deterministic/stochastic; age does not guarantee quality.

Survivor selection (*selecția supraviețuitorilor / selecția generației următoare*) chooses who enters the next generation. It is distinct from parent selection.

Parent selection asks: **Who reproduces?** Survivor selection asks: **Who remains in the population?**

Survivor selection can choose from:

- current population only;
- descendants only;
- current population plus descendants.

In the course, a common formula is: next generation is selected from current individuals and offspring obtained after mutation. It may use fitness only, age only, or a combination of fitness and age. It may be deterministic, stochastic or mixed.

Important mechanisms:

- generational replacement: most or all offspring replace the current generation;
- elitism: best old individual survives if better than all descendants;
- superelitism: strongest deterministic preservation of best individuals;
- steady-state / GENITOR: a small number of descendants replace a small number of weak individuals;
- age-based replacement: older individuals are removed regardless of quality;
- quality-based replacement: best fitness individuals survive.

MCQ traps:

- survivor selection is not necessarily at the beginning of each iteration;
- it is not always random;
- selected survivors are not automatically feasible unless feasibility is enforced;
- age-based selection does not guarantee higher average fitness;
- quality-based or elitist mechanisms can increase or preserve best quality, but not every mechanism guarantees mean improvement.

Supreme memory card

- Survivor selection chooses the next generation after descendants exist.
- May use quality, age, or both; may be deterministic, stochastic, or mixed.
- Elitism preserves best individuals; age-based replacement does not guarantee average quality improvement.

High-yield MCQ overlay from the improved edition

Exam Frequency: High

Survivor-selection answer pattern note: previous MCQs sometimes expect a broad set: it may use current population, descendants, or both; it is applied after new candidates exist; it may be deterministic or stochastic; feasibility is guaranteed only if the pool/repair/constraints enforce it.

Exam trap: Age-based selection does NOT guarantee superior average quality.

Practice exercises and drills

Exercise 13.1 — Survivor Selection Tracing: Generation $P_0 = \{A(10), B(20), C(15), D(25)\}$. After parent selection, recombination, and mutation: offspring = $\{E(30), F(12), G(22), H(18)\}$. Apply each survivor selection mechanism and determine P_1 : (a) Generational replacement (all offspring replace all parents) (b) Elitism: keep best parent, rest from offspring (c) Age-based: remove oldest (A), insert best offspring (d) $(\mu+\lambda)$ selection: select best 4 from $P_0 \cup$ offspring

Exercise 13.2 — GENITOR Analysis: In a GENITOR model ($\lambda=1$), population size 100. How many generations until the entire population is replaced? How does this compare to generational replacement? What are the advantages and disadvantages of GENITOR over generational?

Exercise 13.3 — Age-Based Pitfall: Given population $\{A(50, \text{age}=3), B(40, \text{age}=2), C(30, \text{age}=1), D(60, \text{age}=4)\}$ with age limit = 3. Which individuals survive? Is the average quality of survivors higher or lower than the average quality of the population? Explain why age-based selection doesn't guarantee quality improvement.

Exercise 13.4 — (μ, λ) vs $(\mu+\lambda)$: $\mu=5, \lambda=10$. Current population fitness: [50, 40, 35, 30, 25]. Offspring fitness: [45, 55, 20, 60, 15, 48, 38, 52, 22, 42]. (a) For $(\mu+\lambda)$: select best 5 from all 15. What are their fitness values? (b) For (μ, λ) : select best 5 from offspring only. What are their fitness values? (c) Why does (μ, λ) help escape local optima better than $(\mu+\lambda)$?

CHAPTER 14 — Classical Genetic Algorithm model

Final-edition quick exam handle

Exam frequency: Very high. Canonical GA pattern: 1,3,5,8 = binary representation, FPS selection, low mutation, high recombination.

The canonical/classical GA model is a high-frequency MCQ target. Its features are:

- representation: binary strings;
- population: usually fixed size;
- initialization: random initial population;
- parent selection: fitness-proportionate selection in the classical model;
- crossover: high probability, main variation operator;
- mutation: low probability;
- survivor selection: often age-based/generational replacement in the canonical model;
- repeated evolution until termination by generations, quality threshold, stagnation or similar.

Canonical GA does not mean all modern GAs must be binary. Modern GAs use binary, integer, real-valued and permutation encodings. But if the MCQ says "algorithm genetic classic/canonical," choose the canonical features: binary representation, FPS selection, high recombination, low mutation.

Exam-risk note: one MCQ key in the corpus marks "recombination is rare" as correct in a maximizing GA. This contradicts the canonical GA course material, where high crossover probability is a defining property. Treat that as a corrupted key unless the wording refers to a very special noncanonical algorithm.

Supreme memory card

- Canonical GA: binary representation, FPS parent selection, low mutation, high crossover.
- Canonical GA is not a guarantee of optimality; it is the classical reference model.
- Age-based replacement may be part of canonical/simple generational replacement depending on course wording.

High-yield MCQ overlay from the improved edition

Exam Frequency: Very High — appears 5+ sessions

ANSWER PATTERN: 1, 3, 5, 8

Practice exercises and drills

Exercise 14.1 — Identification: Given the following GA configurations, identify which ones are canonical GA and which are not. Explain why: (a) Real-valued vectors, Gaussian mutation, probability 1 mutation $\rightarrow ?$ (b) Binary strings, FPS selection, $pm=0.01$, $pr=0.8 \rightarrow ?$ (c) Permutations, PMX crossover, swap mutation $\rightarrow ?$ (d) Binary strings, tournament selection, $pm=0.5$, $pr=0.3 \rightarrow ?$

Exercise 14.2 — Parameter Analysis: In a canonical GA with population 100 and chromosome length 50: (a) How many crossover operations are expected per generation with $pr=0.8$? (b) How many mutations are expected with $pm=0.01$? (c) If you increase p_m to 0.5, what happens to the search behavior? Is it still a "canonical" GA? (d) Why does the course emphasize "high crossover, low mutation" for canonical GA?

Exercise 14.3 — Deviation from Canonical: For each deviation from canonical GA, predict whether search performance would improve or degrade (and why): (a) Use tournament instead of FPS (b) Use permutation instead of binary representation (c) Use $pm = pr = 0.5$ (d) Add elitism to survivor selection

CHAPTER 15 — Applications and representation templates

Final-edition quick exam handle

Exam frequency: Medium. Application matching: knapsack=binary; N-Queens/TSP/JSS=permutation; continuous knapsack/portfolio weights=real; transportation may use permutation decoding.

A. 0-1 Knapsack

Problem: choose objects with values and costs so total value is maximized and total cost does not exceed C_{max} .

Representation: binary string, where gene $i = 1$ means object i selected. Fitness: total value, with feasibility condition or penalty for exceeding cost. Mutation: bit flip. Recombination: one-point, multi-point, uniform. Selection: FPS/rank/tournament possible if fitness is positive. Trap: a candidate exceeding C_{max} is infeasible unless repaired or penalized.

B. Continuous knapsack

Representation may be real-valued fractions or quantities. Mutation: real uniform/non-uniform/Gaussian. Recombination: arithmetic. Fitness: value subject to capacity.

C. N-Queens

Problem: place N queens so that no two attack each other. Representation: permutation, often one queen per column and allele = row. This removes row/column conflicts and leaves diagonal conflicts to minimize. Fitness: number of non-attacking pairs or inverse of conflict count. Mutation: swap, inversion, insertion. Recombination: OX/OCX may be used when order interpretation is important; PMX/CX can also be valid permutation operators depending on modeling. Trap: ordinary binary/integer crossover may create duplicate rows unless representation is carefully repaired.

D. Traveling Salesman Problem

Problem: find a tour visiting all cities once and returning to the start with minimum cost. Representation: permutation of cities. Fitness: often $1 / \text{route_cost}$. Mutation: swap, inversion, insertion, scramble. Recombination: ECX is natural

for preserving edges; PMX/OX/CX also appear. Trap: TSP depends strongly on adjacency/edges; ECX is the operator to remember for common-edge preservation.

E. Job-shop/activity scheduling

Problem: schedule operations on machines subject to precedence and machine availability. Representation: permutations of operations or concatenations of machine-specific permutations. Fitness: makespan, lateness, cost or inverse of schedule cost. Mutation: swap/insertion/inversion in operation order. Recombination: order-preserving or permutation-specific operators. Trap: genotype order may be decoded into a feasible schedule; feasibility can be handled by decoding.

F. Transportation/resource allocation/loading

Representation: often a permutation specifying allocation order, or numeric quantities. Fitness: total transport cost, capacity use, response time or utility. Mutation/recombination depend on whether the chromosome is a permutation or numeric vector. Trap: if any permutation can be decoded into a feasible allocation order, the genotype problem becomes effectively unconstrained.

G. Maze, puzzle, placement, aircraft/purchase examples

Maze and path problems often use binary or integer choices for movement. Puzzle/placement problems may use permutations. Aircraft purchase/resource examples use integer strings. Fitness combines objective value and constraints.

H. Portfolio optimization

Representation: real-valued vector of weights or mixed integer/real decisions. Fitness: risk-return objective, sometimes multi-objective. Mutation: real perturbation. Recombination: arithmetic. Trap: constraints such as weights summing to 1 require repair/normalization or penalties.

Supreme memory card

- Match representation to problem: Boolean subset -> binary; quantities/categories -> integer; continuous -> real; order/route/schedule -> permutation.
- TSP/N-Queens/scheduling require permutation logic and special operators.
- Economic applications include JSS, portfolio optimization, transportation/resource allocation.

Practice exercises and drills

Exercise 15.1 — Full GA Design: Given a new problem (assigning 8 nurses to 8 shifts where each nurse has different efficiency at each shift, maximize total efficiency): (a) Choose the representation and justify. (b) Design the fitness function. (c) Choose mutation and recombination operators and justify. (d) Choose parent and survivor selection mechanisms. (e) Define the termination condition.

Exercise 15.2 — N-Queens Fitness Calculation: For $P = [2,4,1,3]$ on a 4×4 board, calculate $f(p)$ = number of non-attacking pairs. List all $C(4,2) = 6$ pairs and check each for diagonal attacks. What is $f(p)$? What is the maximum possible $f(p)$ for $n=4$?

Exercise 15.3 — TSP Cost Calculation: Given 4 cities with cost matrix:

	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0

Calculate $\text{cost}(p)$ and fitness for permutation $p = [1,3,2,4]$.

Exercise 15.4 — Representation Comparison: For the same N-Queens problem, compare permutation representation vs. binary representation (n^2 bits, one per square). Compare: search space size, constraint handling, operator choice, and likelihood of finding a solution in the first generation.

CHAPTER 16 — Evolution Strategies

Final-edition quick exam handle

Exam frequency: High. ES pattern: real vectors, mutation probability 1, random/uniform parent selection, self-adaptive sigma; ES-2M = single stochastic continuous candidate.

Evolutionary Strategies (ES, *strategii evolutive*) are a class of evolutionary algorithms typically used for continuous spaces. Genotypes are real-number vectors, often containing both solution variables and strategy parameters. ES differs from classical GA because mutation is the main variation operator; crossover may exist but is not the central mechanism.

Core ES features:

- representation: real-valued vectors;
- parent selection: often random/uniform;
- mutation: Gaussian perturbation;
- mutation step size sigma (`sigma`) controls mutation magnitude;
- next-generation selection: (`mu`, `lambda`) or (`mu` + `lambda`);
- self-adaptation: strategy parameters evolve during the search.

ES-2M / two-member evolutionary strategy

This is a simple ES with one current candidate and one mutated descendant.

1. Choose initial point x randomly.
2. Generate Gaussian noise $z_i \sim N(0, \text{sigma})$ for each component.
3. Create descendant $y = x + z$.
4. If $f(y)$ is better than $f(x)$, replace x with y ; otherwise keep x .
5. Repeat until termination.

This method is stochastic and suited to continuous spaces. It may reach local or global optima depending on landscape and parameters.

Mutation probability in ES

MCQs repeatedly test: in ES, mutation is applied with probability 1. This means mutation is the core production mechanism, not an occasional low-probability afterthought like classical GA mutation.

Self-adaptation

In ES, sigma can be encoded in the chromosome and evolved. If sigma is too large, search jumps over good regions; if too small, search stagnates. The classic 1/5 success rule adapts sigma according to the proportion of successful mutations: if success rate is high, increase search radius; if low, decrease it.

(`mu` + `lambda`) versus (`mu`, `lambda`)

- (`mu` + `lambda`) : select survivors from both parents and offspring; elitist because parents may survive.
- (`mu`, `lambda`) : select survivors only from offspring; less elitist and can avoid preserving stale parents.

MCQ traps:

- ES is not binary-vector canonical GA.
 - ES does not use PMX/OCX as standard core operators.
 - ES mutation is not swap mutation; it is usually Gaussian perturbation.
 - ES mutation is performed iteratively and immediately when children/offspring are available.
-

Supreme memory card

- ES usually uses real-valued vectors and mutation with probability 1.
- Recombination can be local/global and discrete/intermediate.
- $(\mu + \lambda)$ selects from parents+offspring; (μ, λ) selects from offspring only; self-adaptation evolves sigma.

High-yield MCQ overlay from the improved edition

Exam Frequency: High

ANSWER PATTERN: 2, 4, 5, 6 — single candidate, Gaussian per component, continuous, stochastic.

Practice exercises and drills

Exercise 16.1 — ES-2M Trace: Maximize $f(x) = -(x-3)^2$. Start $x_0 = 1$, $\sigma = 0.5$. (a) Generate $z \sim N(0, 0.5)$. Say $z = 0.7$. Compute $y = 1 + 0.7 = 1.7$. (b) Evaluate: $f(1) = -(1-3)^2 = -4$. $f(1.7) = -(1.7-3)^2 = -1.69$. (c) Since $f(1) \leq f(1.7)$, accept: $x_1 = 1.7$. (d) Repeat for 3 more iterations with $z = -0.3, 1.2, -0.8$. Track the trajectory.

Exercise 16.2 — ES σ Update: Given $\sigma = 1.0$, $\tau = 0.5$, $r\tau = 0.3$: (a) Calculate $\sigma' = \sigma \cdot \exp(r\tau) = 1.0 \cdot \exp(0.3)$. (b) If $\sigma' < \epsilon\sigma = 0.01$, what happens? (c) Now mutate $x = [2.0, 5.0]$ using σ' and $f_{x1} \sim N(0, \sigma') = 0.4$, $f_{x2} = -0.6$. (d) What is the new chromosome (x', σ') ?

Exercise 16.3 — (μ, λ) vs $(\mu + \lambda)$: $\mu=3, \lambda=5$. Current: [40, 35, 30]. Offspring: [50, 25, 45, 20, 55]. (a) $(\mu + \lambda)$: select best 3 from all 8. Result? (b) (μ, λ) : select best 3 from 5 offspring. Result? (c) Why does (μ, λ) prevent survival of weak individuals better?

Exercise 16.4 — Conceptual: Explain why ES mutation probability is 1 (always applied) while GA mutation probability is low. How does this relate to ES being "mutation-centered" vs GA being "crossover-centered"? What would happen if you used GA's low mutation probability in ES?

CHAPTER 17 — Memetic and hybrid algorithms

Final-edition quick exam handle

Exam frequency: Medium. Memetic algorithms = EA plus local improvement; valid improvement methods include hillclimbing, ES-2M, exact/heuristic/gradient methods.

A memetic algorithm is an evolutionary algorithm combined with local improvement. It uses population-level exploration plus individual-level refinement. The local improvement method can be Hillclimbing, an ES-2M-like local strategy, a heuristic, an exact method, or a gradient method where applicable.

The key exam idea is: memetic algorithms are hybrid. The EA generates and selects candidate solutions; the local search improves some individuals before reinserting them. This can improve exploitation but increases computational cost and may reduce diversity if overused.

Valid improvement mechanisms, depending on problem and course context:

- Hillclimbing;
- local search heuristics;

- ES-style perturbation/refinement;
- exact methods for subproblems;
- gradient-based methods in continuous differentiable contexts.

Invalid trap answers are generic programming utilities or mechanisms that do not improve candidate solutions. The improvement mechanism must operate on solutions or neighborhoods, not on display, memory allocation, or unrelated validation.

Supreme memory card

- Memetic algorithms = population-based EA plus local improvement.
- Hybridization can improve exploitation but increases computational cost.
- Local search can be applied to all individuals, selected individuals, or only best candidates.

High-yield MCQ overlay from the improved edition

■ **ANSWER PATTERN: 2, 4, 6, 7, 9** (Hillclimbing, ES-2M, exact, heuristic, gradient)

Practice exercises and drills

Exercise 17.1 — Classification: For each technique, classify as valid or invalid for memetic improvement: (a) Simulated annealing as local search (b) Running a greedy algorithm on each individual (c) Using ECX crossover (d) Running 5 steps of gradient descent on real-valued individuals

Exercise 17.2 — Design: Design a memetic algorithm for TSP. Specify: (a) base EA, (b) local improvement method, (c) when to apply local improvement (every generation? only best individual?), (d) computational cost trade-off.

Exercise 17.3 — Conceptual: Why is ECX listed as invalid in the MCQ context but valid as a crossover operator? Explain the distinction between "improvement mechanism" and "variation operator."

CHAPTER 18 — Exam traps and final mastery rules

Final-edition quick exam handle

Exam filter chapter. Convert traps into flashcards: always true, usually true, false distractors, and exam-risk contradictions.

Always true / high-confidence rules

1. EC is inspired by natural biological evolution.
2. Natural evolution is modeled as generate-test / trial-and-error.
3. Environment corresponds to search/solution space.
4. Individual/chromosome/genotype corresponds to encoded candidate solution.
5. Phenotype is the decoded real solution.
6. Fitness/adaptation corresponds to solution quality.
7. EA components include representation, fitness, population, parent selection, variation, survivor selection, initialization, termination.
8. Fitness evaluates each candidate/individual.
9. Fitness is the basis of selection.
10. Fitness does not evaluate the algorithm's speed.
11. Fitness is not randomly changed at each iteration.
12. The initial population is created before evolution proper.

13. The initial population is usually random.
14. A population is a multiset of genotypes.
15. Individuals are static; populations evolve.
16. Diversity can mean distinct genotypes, phenotypes or fitness values.
17. Termination condition is mandatory.
18. Valid termination includes max generations, target quality, stagnation and diversity threshold.
19. Binary representation is canonical for classical GA.
20. Integer representation is suitable when genes have more than two values.
21. Real representation is for continuous domains.
22. Permutations are for ordering/assignment problems.
23. Permutations require special operators.
24. Bit flip is binary mutation.
25. Creep/fluaj is an integer/numeric mutation.
26. Gaussian perturbation is central in ES.
27. Swap, insertion, inversion and scramble preserve permutations if correctly applied.
28. Recombination in GA is usually binary.
29. Classical GA uses high crossover probability.
30. Classical GA uses low mutation probability.
31. Recombination is performed iteratively, not once.
32. Mutation may produce infeasible individuals.
33. Survivor selection is not the same as parent selection.
34. Parent selection selects reproduction candidates.
35. Survivor selection selects the next generation.
36. FPS probability is proportional to positive fitness.
37. FPS may cause premature convergence.
38. SUS approximates the selection distribution better than simple roulette.
39. Simple roulette is spun k times for k selections.
40. Multi-arm roulette/SUS can complete selection with one spin.
41. Tournament selection is not necessarily deterministic.
42. Rank selection uses rank, not raw fitness.
43. PMX, OX, CX and ECX are permutation crossover operators.
44. OX preserves relative order.
45. CX preserves absolute positions through cycles.
46. ECX preserves edges/adjacencies.
47. TSP uses permutation representation.
48. N-Queens can use permutation representation.
49. Knapsack 0-1 uses binary representation.
50. ES uses real vectors and mutation-centered search.
51. In ES, mutation is commonly applied with probability 1.
52. ES self-adapts sigma.
53. Hillclimbing works on one current point.
54. Hillclimbing usually finds a local optimum.
55. Multi-start Hillclimbing improves chances of better results.
56. Memetic algorithms combine EA with local improvement.

Usually true rules

1. Parent selection is usually stochastic/nondeterministic.
2. Survivor selection may be deterministic, stochastic or mixed.
3. Initialization is usually uniform random if no reason otherwise.

4. Recombination is the main GA variation operator.
5. Mutation maintains diversity and helps escape local optima.
6. Elitism preserves the best old individual.
7. Generational replacement propagates most or all offspring.
8. Steady-state/GENITOR replaces only a few weak individuals.

False distractor rules

1. Fitness evaluates the algorithm: false.
2. Fitness evaluates memory consumption: false.
3. Fitness must be selected randomly: false.
4. Population initial is regenerated every cycle: false.
5. Binary representation always gives optimum: false.
6. Representation is independent of the problem: false.
7. Permutation representation is not used: false.
8. Permutation representation cannot use mutation/recombination: false; it needs special ones.
9. Mutation is used only in constrained problems: false.
10. Recombination is used only in unconstrained problems: false.
11. Tournament selection is always deterministic: false.
12. Hillclimbing always finds the global optimum: false.
13. ES uses binary vectors as standard: false.
14. ES mutation is swap mutation: false.
15. Ordinary arithmetic recombination is valid for permutations: false.
16. Age-based selection guarantees better mean quality: false.

High-risk confusing pairs

- Fitness function vs selection mechanism: fitness measures; selection chooses.
- Parent selection vs survivor selection: reproduction pool vs next population.
- Genotype vs phenotype: encoded representation vs decoded solution.
- Mutation vs recombination: one individual locally changed vs parents combined.
- OX vs CX: order preservation vs position-cycle preservation.
- PMX vs ECX: mapping/segment repair vs edge adjacency preservation.
- GA vs ES: crossover-centered binary canonical model vs mutation-centered real-vector model.
- Hillclimbing vs EA: one-point local search vs population evolution.

Supreme memory card

- Treat answer-key patterns as previous-exam signals, not universal theory when keys conflict.
- When course theory and key conflict: learn theory, then remember exam-risk expectation.
- Highest traps: fitness, survivor selection, permutation validity, ES mutation, canonical GA, recombination order.

Practice exercises and drills

Exercise 18.1 — Trap Identification: For each statement, mark ALWAYS TRUE, USUALLY TRUE, or FALSE. If false, explain why:

(a) "The fitness function provides the basis for selection." (b) "Hillclimbing is guaranteed to find the global optimum." (c) "Permutation crossover operators preserve the permutation property." (d) "Initial population is generated using normal distribution." (e) "ES uses fitness-proportionate parent selection." (f) "Survivor selection can be deterministic."

Exercise 18.2 — Romanian Translation: Translate and explain the exam significance of: "Necesită operatori de variație special definiți", "Se aplică asupra descendenților produși de operația de recombinare", "Poate să producă indivizi nefezabili".

CHAPTER 19 — Operator calculation workshop

Final-edition quick exam handle

Calculation chapter. Solve by hand: PMX, CX, scramble, inversion, arithmetic crossover, FPS/SUS, ES sigma update.

How to solve PMX MCQs

1. Write parents with positions numbered.
2. Mark the two cut positions.
3. Copy the segment from the first parent into the child.
4. List mapping pairs between the two parent segments.
5. Fill outside positions from the other parent.
6. When a value is already in the child, follow mapping until a free value is found.
7. Check: every allele appears exactly once.

Quick validation: if a proposed PMX child has duplicates or missing alleles, eliminate it immediately.

How to solve CX MCQs

1. Start at position 1.
2. Take value from parent 2 at that position.
3. Find that value's position in parent 1.
4. Repeat until you return to the starting position: this is one cycle.
5. Start another cycle at the first unused position.
6. Alternate cycles between parents.
7. Compare with answer options.

For the repeated exam-style parents: $X = [6, 1, 8, 10, 5, 7, 9, 3, 4, 2]$ $Y = [9, 8, 7, 3, 6, 1, 5, 10, 4, 2]$

Cycles are:

- positions 1,7,5
- positions 2,3,6
- positions 4,8
- position 9
- position 10

Alternating cycles from X, Y, X, Y, X gives: $C1 = [6, 8, 7, 10, 5, 1, 9, 3, 4, 2]$ $C2 = [9, 1, 8, 3, 6, 7, 5, 10, 4, 2]$

If an MCQ key claims another option, suspect corruption.

How to solve OX/OCX MCQs

1. Copy the middle segment from parent 1.
2. Read parent 2 cyclically after the second cut.
3. Skip values already in the child.
4. Fill empty positions in cyclic order.
5. The final child must preserve the copied segment and preserve relative order of the remaining parent-2 values.

Use OX when the question says the permutation represents "order of events" (*ordinea apariției evenimentelor*).

How to solve ECX concept questions

Look for keywords:

- common relationships (*relații comune*);
- edges (*muchii*);
- adjacency (*adiacență*);
- TSP route links.

Answer: ECX / edge crossover.

How to solve inversion mutation

Find the changed interval and reverse it. Outside genes stay fixed. Original [1, 2, 3, 4, 5, 6]; positions 2..5 -> [1, 5, 4, 3, 2, 6].

How to solve scramble/amestec mutation

Find the smallest and largest differing positions. The elements inside must be the same set but in shuffled order. Outside positions are identical.

Example: Original [7, 6, 12, 14, 3, 10, 8, 15, 11, 5, 4, 1, 13, 2, 9] Mutated [7, 14, 13, 12, 1, 15, 2, 8, 6, 3, 11, 5, 10, 4, 9]

Positions 2..14 differ, while positions 1 and 15 remain the same. The segment has the same elements in a different order, so the scramble interval is 2 and 14.

How to solve insertion mutation

One element is removed and reinserted elsewhere. The sequence between the old and new position shifts. There are no duplicates and no missing values.

How to solve creep/fluaj mutation

Look for a small numeric change in one or more integer genes. Original [6, 1, 8, 10, 5]; possible creep result [6, 1, 9, 10, 5]. A permutation-like rearrangement is not creep; a bit flip is not creep.

How to identify valid descendants quickly

- Binary child: must contain only 0/1.
- Integer child: each gene must be in allowed integer range.
- Real child: values must be real and inside bounds if enforced.
- Permutation child: must contain every allele exactly once.
- If multiple choices are valid, use the operator's preservation principle: OX order, CX cycles/positions, ECX edges, PMX mapping.

Supreme memory card

- Always verify permutation descendants: same length, same elements, no duplicates, no missing values.
- For PMX/CX/OX, write positions under parents before calculating.
- For mutation, compare original and result to infer changed segment/positions.

Additional enhanced workshop/checklist

19.1 PMX Full Worked Example

Problem: P1 = {6,3,11,7,14,8,5,15,1,2,4,13,9,10,12}, P2 = {7,1,15,13,2,14,6,10,12,11,4,8,3,9,5}, cuts at positions 4 and 8.

Step 1: Copy segment from P1 P1[4..8] = {7, 14, 8, 5, 15} C1 = [?, ?, ?, **7, 14, 8, 5, 15**, ?, ?, ?, ?, ?, ?]

Step 2: Build mapping (P1 segment ↔ P2 segment) P2[4..8] = {13, 2, 14, 6, 10} Mapping: 7↔13, 14↔2, 8↔14, 5↔6, 15↔10

Step 3: Fill remaining from P2, resolving conflicts

- Pos 1: P2[1]=7 → 7 in segment → map 7→13 → 13 not in C1 → C1[1]=**13**
- Pos 2: P2[2]=1 → 1 not in C1 → C1[2]=**1**
- Pos 3: P2[3]=15 → 15 in segment → map 15→10 → 10 not in C1 → C1[3]=**10**
- Pos 9: P2[9]=12 → 12 not in C1 → C1[9]=**12**
- Pos 10: P2[10]=11 → 11 not in C1 → C1[10]=**11**
- Pos 11: P2[11]=4 → 4 not in C1 → C1[11]=**4**
- Pos 12: P2[12]=8 → 8 in segment → map 8→14 → 14 in segment → map 14→2 → 2 not in C1 → C1[12]=**2**
- Pos 13: P2[13]=3 → 3 not in C1 → C1[13]=**3**
- Pos 14: P2[14]=9 → 9 not in C1 → C1[14]=**9**
- Pos 15: P2[15]=5 → 5 in segment → map 5→6 → 6 not in C1 → C1[15]=**6**

Result: C1 = {13, 1, 10, 7, 14, 8, 5, 15, 12, 11, 4, 2, 3, 9, 6} ✓

19.2 CX Full Worked Example

Problem: X = [6,1,8,10,5,7,9,3,4,2], Y = [9,8,7,3,6,1,5,10,4,2]

Find cycles:

- Start pos 1: X[1]=6 → Y has 6 at pos 5 → X[5]=5 → Y has 5 at pos 7 → X[7]=9 → Y has 9 at pos 1 → back to start. **Cycle 1: {1,5,7}**
- Next uncovered pos 2: X[2]=1 → Y has 1 at pos 6 → X[6]=7 → Y has 7 at pos 3 → X[3]=8 → Y has 8 at pos 2 → back. **Cycle 2: {2,3,6}**
- Next uncovered pos 4: X[4]=10 → Y has 10 at pos 8 → X[8]=3 → Y has 3 at pos 4 → back. **Cycle 3: {4,8}**
- Remaining pos 9: X[9]=4, Y[9]=4 (same) → pos 10: X[10]=2, Y[10]=2 (same). **Cycle 4: {9,10}**

Build C1: Cycle 1+3 from X, Cycle 2+4 from Y:

- Pos 1,5,7 (Cycle 1): X values → 6, 5, 9
- Pos 4,8 (Cycle 3): X values → 10, 3
- Pos 2,3,6 (Cycle 2): Y values → 8, 7, 1
- Pos 9,10 (Cycle 4): Y values → 4, 2

C1 = [6, 8, 7, 10, 5, 1, 9, 3, 4, 2] ✓

19.3 Inversion Mutation

Problem: X = [6,1,8,10,5,7,9,3,4,2], positions (2,7)

Segment [2..7]: [1, 8, 10, 5, 7, 9] Reversed: [9, 7, 5, 10, 8, 1] Result: **[6, 9, 7, 5, 10, 8, 1, 3, 4, 2] ✓**

19.4 Scramble Mutation

Problem: X = [7,6,12,14,3,10,8,15,11,5,4,1,13,2,9] → mutated: [7,14,13,12,1,15,2,8,6,3,11,5,10,4,9]

Method to find boundaries:

- Pos 1: 7=7 (same) → outside
- Pos 2: 6→14 (changed) → start of segment
- Pos 14: 2→4 (changed)
- Pos 15: 9=9 (same) → outside
- Segment: positions 2-14 (or 2-13, or 1-14, or 1-15 depending on MCQ options)

- Original segment: {6,12,14,3,10,8,15,11,5,4,1,13,2}
- Mutated segment: {14,13,12,1,15,2,8,6,3,11,5,10,4}
- Same multiset? {1,2,3,4,5,6,8,10,11,12,13,14,15} ✓

19.5 Arithmetic Crossover with Numbers

Problem: P1 = [1.4, 2.5, 0.1, 5.8, 3.4], P2 = [4.5, 6.1, 3.6, 3.5, 2.4], $\alpha=0.3$

Simple arithmetic at position 4: C1[4] = $0.3 \times 5.8 + 0.7 \times 3.5 = 1.74 + 2.45 = 4.19$ C2[4] = $0.7 \times 5.8 + 0.3 \times 3.5 = 4.06 + 1.05 = 5.11$

For integers with rounding: P1 = [10, 20, 30, 40], P2 = [20, 10, 40, 20], $\alpha=0.3$ C1[3] = $\text{round}(0.3 \times 30 + 0.7 \times 40) = \text{round}(9+28) = 37$ C2[3] = $\text{round}(0.7 \times 30 + 0.3 \times 40) = \text{round}(21+12) = 33$

19.6 ES Mutation Worked Example

Problem: Maximize $f(x,y) = -(x-3)^2 - (y-4)^2$. Current: $x = [1.0, 2.0]$, $\sigma = 0.5$, $\tau = 0.5$.

Step 1: Mutate $\sigma_{r_{\tau}} \sim N(0,1) = 0.3$ (given) $\sigma' = 0.5 \times \exp(0.3) = 0.5 \times 1.3499 = 0.675$

Step 2: Mutate solution $fx1 \sim N(0, 0.675) = 0.4$, $fx2 \sim N(0, 0.675) = -0.6$ $x' = [1.0 + 0.4, 2.0 + (-0.6)] = [1.4, 1.4]$

Step 3: Evaluate $f([1.0, 2.0]) = -(1-3)^2 - (2-4)^2 = -4 - 4 = -8$ $f([1.4, 1.4]) = -(1.4-3)^2 - (1.4-4)^2 = -2.56 - 6.76 = -9.32$

Since $f(\text{parent}) > f(\text{offspring})$ ($-8 > -9.32$), **reject offspring**. $x_{\text{next}} = [1.0, 2.0]$.

19.7 OCX with Wrap-Around

Problem: P1 = [1,2,3,4,5,6,7,8,9], P2 = [9,3,7,8,2,6,5,1,4], cuts at 3 and 7.

Step 1: Copy P1[3..7] = [3,4,5,6,7] Child: [, 3,4,5,6,7, ,] Used: {3,4,5,6,7}

Step 2: Remaining from P2 in order: P2 = [9,3,7,8,2,6,5,1,4] Skip 3,7,6,5 (used). Remaining: 9, 8, 2, 1, 4

Step 3: Fill from position 8 onwards, then wrap to 1: Pos 8: 9, Pos 9: 8, Pos 1: 2, Pos 2: 1

Child = [2, 1, 3, 4, 5, 6, 7, 9, 8]

CHAPTER 20 — Final coverage checklist

Final-edition quick exam handle

Final checklist chapter. Memorize exact answer patterns and high-risk pairs; use it as the last 30-minute review sheet.

Can I answer the previous-exam MCQs?

You are covered if you can answer these groups:

- EC inspiration and mapping between biology and problem solving.
- The three problem classes: optimization, modeling/system identification, simulation.
- EA components: representation, fitness, population, parent selection, variation, survivor selection, initialization, termination.
- Representation theory: genotype, phenotype, gene, allele, encoding, decoding.
- Binary, integer, real and permutation representations.
- Fitness traps: candidate quality, not algorithm speed/resources.
- Population traps: multiset, static individual, evolving population, diversity.
- Initialization and termination criteria.

- Hillclimbing: one point, neighbors, local optimum, repeated starts.
- Mutation operators: bit flip, creep, random reset, uniform, non-uniform, Gaussian, swap, insertion, inversion, scramble.
- Recombination operators: one-point, multi-point, uniform, arithmetic.
- Permutation crossover: PMX, OX/OCX, CX, ECX.
- Parent selection: FPS, windowing, sigma scaling, rank, roulette, SUS, tournament.
- Survivor selection: generational, steady-state, elitism, age/quality selection.
- Canonical GA: binary, FPS, high crossover, low mutation.
- Applications: knapsack, continuous knapsack, N-Queens, TSP, JSS, transportation, placement, portfolio.
- ES: real vectors, Gaussian mutation, sigma, ES-2M, self-adaptation, $(\mu + \lambda)$ and (μ, λ) .
- Memetic/hybrid algorithms: EA plus local improvement.

Most frequent MCQ concepts

The corpus heavily emphasizes mutation, recombination, selection, population, permutation operators, ES, Hillclimbing and representation choice. Calculation questions are especially concentrated around PMX, CX, scramble mutation and operator identification.

High-risk confusing pairs to memorize

- fitness vs quality vs objective;
- parent selection vs survivor selection;
- roulette vs SUS;
- generational vs steady-state population model;
- PMX vs OX vs CX vs ECX;
- GA mutation probability low vs ES mutation probability 1;
- local optimum vs global optimum;
- binary/int/real/permutation representations.

Things to memorize exactly

- Optimization/modeling/simulation triad.
- EA components list.
- Canonical GA features.
- Fitness evaluates each candidate.
- Population is the unit of evolution.
- Initial population is generated before evolution.
- Permutation representation needs special operators.
- FPS drawback: premature convergence.
- SUS is closest to the desired probability distribution.
- ECX preserves edges; OX preserves order; CX preserves cycles/positions.
- ES uses Gaussian mutation and self-adaptive sigma.

Things to understand procedurally

- Compute FPS probabilities.
- Simulate roulette versus SUS conceptually.
- Execute bit flip, creep, inversion, insertion and scramble.
- Execute PMX with mapping repair.
- Execute OX with cyclic fill.
- Execute CX by building cycles.
- Recognize infeasible offspring and choose repair/parent-copy/selection-handling logic.

- Decode application problems into representation + fitness + operators.

Consolidated Exam-Risk Notes

1. **Permutation representation.** Some keys mark only "needs special variation operators," while others also mark "does not allow duplicate genes." The theoretical answer is that a permutation has no duplicates and needs special mutation/recombination.
2. **CX calculation.** At least one repeated CX item appears to have a corrupted correct option. Use cycle construction, not the printed key, when reasoning.
3. **Integer arithmetic recombination.** One key says arithmetic recombination for integer strings requires same parity; another says it is possible with rounding. The theoretical answer is: possible with rounding and validity checks.
4. **Hillclimbing.** Some keys incorrectly classify Hillclimbing as self-adaptive/evolutionary. The theoretical course framing is local search on one point; it can be embedded into memetic algorithms.
5. **Recombination frequency in GA.** Classical GA has high crossover probability; any key saying recombination is rare is inconsistent with canonical GA unless a special noncanonical case is intended.
6. **Survivor selection.** Some MCQ versions give narrow answers; others give broader correct sets. The theory: it may involve current population, offspring, or both, and may use age, quality, or both.
7. **Parent selection deterministic/nondeterministic.** The safest theory: generally nondeterministic/stochastic, but deterministic implementations exist. If the wording says "in general," choose nondeterministic.

Supreme memory card

- Use Chapter 20 as the final pre-exam memorization grid.
- Prioritize Chapters 11, 12, 13, 16, 18, 19 for calculation and trap resistance.
- Do not memorize corrupted answers blindly; mark them as exam-risk.

Additional enhanced workshop/checklist

20.1 Can I Answer the Previous-Exam MCQs?

Topic	Covered?	Answer Pattern
EC inspiration	YES	Biological evolution
Problem classes	YES	1, 4, 6
EA components	YES	1,3,5,7,10,12,14,15
Representation choice	YES	Depends on problem + influences quality
Fitness function	YES	9 or 2,4
Initial population	YES	Before evolution; random; often/course-key uniform; once
Termination	YES	Mandatory + iteration control
Hillclimbing	YES	1, 2, 5, 7
Canonical GA	YES	1, 3, 5, 8
ES-2M	YES	2, 4, 5, 6
Survivor selection	YES	b,d,e,f,k
Recombination	YES	e,f,g,h,j

FPS disadvantage	YES	Premature convergence
SUS property	YES	Closest to distribution
Tournament property	YES	Does NOT respect distribution
Real-valued operators	YES	5,6,14,15,16,18,19,20
Invalid selection	YES	Ruleta complicata, FPS cu alfa
Invalid problem class	YES	Big-data, validation, memory, vehicles
PMX calculation	YES	Worked in Ch 19
CX calculation	YES	Worked in Ch 19
Inversion	YES	Worked in Ch 19
Scramble	YES	Worked in Ch 19
Arithmetic crossover	YES	Worked in Ch 19
ES mutation	YES	Worked in Ch 19
Memetic methods	YES	2,4,6,7,9

20.2 Most Frequent MCQ Concepts (Priority Order)

1. **Problem classes (1,4,6)** — 10+ appearances
2. **EA components (1,3,5,7,10,12,14,15)** — 8+ appearances
3. **Fitness evaluates each candidate** — 6+ appearances
4. **Representation depends on problem + influences quality** — 6+ appearances
5. **Canonical GA (1,3,5,8)** — 5+ appearances
6. **Hillclimbing (1,2,5,7)** — 5+ appearances
7. **Permutation needs special operators** — 5+ appearances
8. **Survivor selection properties** — 5+ appearances
9. **PMX calculation** — 4+ appearances
10. **Recombination properties** — 4+ appearances
11. **ES mutation = probability 1** — 4+ appearances
12. **ES-2M properties** — 3+ appearances
13. **FPS premature convergence** — 3+ appearances
14. **Tournament doesn't respect distribution** — 3+ appearances

20.3 High-Risk Confusing Pairs

Pair	Distinction
Parent vs Survivor selection	Parent → reproduction pool; Survivor → next generation
Fitness vs Selection	Fitness evaluates; Selection chooses
GA vs ES mutation	GA: low prob; ES: prob 1
PMX vs ECX	PMX: adjacency (TSP); ECX: edges
CX vs OCX	CX: absolute position cycles; OCX: relative order
Individual vs Population	Individual static; Population evolves

Optimization vs Big-data	Valid class vs NOT valid
FPS vs Tournament	FPS respects distribution; Tournament doesn't
SUS vs Simple roulette	SUS: one spin; Simple: tz spins
(μ, λ) vs $(\mu + \lambda)$	(μ, λ) : descendants only; $(\mu + \lambda)$: parents + descendants
Generational vs Steady-state	Generational: $\lambda = \mu$ full; Steady-state: $\lambda \ll \mu$ partial

20.4 Things to Memorize Exactly (No Understanding Needed)

Answer Patterns:

1. EA components: **1, 3, 5, 7, 10, 12, 14, 15**
2. Problem classes: **1, 4, 6**
3. Canonical GA: **1, 3, 5, 8**
4. Hillclimbing: **1, 2, 5, 7**
5. ES-2M: **2, 4, 5, 6**
6. Quality function: **2, 4**
7. Survivor selection: **b, d, e, f, k**
8. Recombination: **e, f, g, h, j**
9. Real-valued operators: **5, 6, 14, 15, 16, 18, 19, 20**
10. Memetic methods: **2, 4, 6, 7, 9**

Facts:

1. SUS → closest to probability distribution
2. Tournament → does NOT respect distribution
3. Multi-arm roulette → rotated **once**
4. Single-arm roulette → rotated **tz times**
5. FPS disadvantage → **premature convergence**
6. ES mutation probability → **1**
7. Invalid selection → **"Ruleta complicata"**
8. Invalid distribution → **"FPS cu alfa scalare"**
9. Fitness evaluates → **each candidate** (NOT algorithm)
10. Decoding → **mandatory after best extraction**

20.5 Things to Understand Procedurally

1. How to execute PMX (copy segment → map → fill → resolve)
2. How to execute CX (find cycles → alternate parents)
3. How inversion works (reverse segment)
4. How scramble works (shuffle segment, find boundaries)
5. How arithmetic crossover works (blend with α , round for integers)
6. How ES σ updates (lognormal: $\sigma' = \sigma \cdot \exp(r_{\tau})$)
7. How fitness transforms for minimization ($-f$ or $1/f$)
8. How to match representation to problem type
9. How to identify which selection mechanism is being described
10. How to verify if an MCQ answer pattern matches the question

SUPREME V3 FINAL RISK MATRIX

Risk area	Theory-safe answer	Exam-safe behavior
Initial population	Generated before evolution; usually random; often uniform; constrained problems may enforce feasibility	Choose before evolution + random; choose uniform when explicitly listed as course/key pattern; avoid saying regenerated each iteration
Fitness vs selection	Fitness evaluates each candidate; selection mechanisms use fitness	If statement says fitness "selects" individuals, treat as false unless wording clearly means selection uses fitness
Parent selection	Generally stochastic/nondeterministic and iterative; uses current evaluated population	If wording says "in general," choose nondeterministic/stochastic; note deterministic implementations are possible but not the exam default
Survivor selection	Chooses next generation; can use parents, offspring, or both; can use age, quality, or both	Do not assume always random or always quality-improving; age-based selection can lower average quality
Feasible individuals	Feasibility can be enforced by representation, repair, filtering, penalties, or selection	Do not claim all mutation/recombination always produce feasible individuals
Permutations	No duplicates and no missing values; special operators required	For MCQ, "requires special operators" is almost always the safest option
PMX vs ECX	PMX maps/repairs a copied segment; ECX preserves edge/adjacency relations	If wording says common edges/adjacency, prefer ECX; if course application says TSP/JSS with PMX, follow that wording
CX calculations	Build value-position cycles and alternate cycles	If printed key disagrees with cycle construction, mark as corrupted-key risk
Integer arithmetic crossover	Possible with rounding and validity/feasibility checks	Reject parity-only rule unless the exact MCQ key is known to be corrupted/peculiar
Hillclimbing	Local search on one current point; may be restarted; usually local optimum	Do not classify as an EA in theory; remember some corrupted keys may do so
Canonical GA	Binary representation, FPS, low mutation, high crossover	Reject "high mutation" and "low/rare recombination" unless special noncanonical wording is present
ES	Real vectors, mutation-centered, mutation probability 1, self-adaptation of sigma	ES is not binary canonical GA; recombination can be local/global and discrete/intermediate

FINAL SUPREME 7-DAY STUDY PLAN

Day 1 — Foundations and components

Read Chapters 1-4. Memorize the mapping between biological evolution and EA, the three problem classes, and the eight EA components. Drill the false statements about fitness, representation and population.

Day 2 — Representations and fitness

Read Chapters 5-7. Make a one-page table: binary, integer, real, permutation. For each, write valid mutation, valid crossover, and common trap.

Day 3 — Mutation and recombination

Read Chapters 9-11. Manually solve at least two examples each of bit flip, creep, swap, insertion, inversion, scramble, one-point crossover, PMX, OX and CX.

Day 4 — Selection mechanisms

Read Chapters 12-13. Calculate FPS probabilities, cumulative roulette intervals, SUS pointers and tournament outcomes. Mark survivor-selection traps separately.

Day 5 — Canonical GA, ES and memetic algorithms

Read Chapters 14, 16 and 17. Memorize canonical GA properties and the ES differences: mutation probability 1, real vectors, local/global recombination, self-adaptation, $(\mu+\lambda)$ vs (μ,λ) .

Day 6 — Exam traps

Read Chapter 18 slowly. Convert each trap into a flashcard. Separate theory-safe answers from exam-risk patterns.

Day 7 — Calculation workshop and final checklist

Work Chapter 19 by hand and keep Chapter 20 open. Redo every permutation calculation until you can detect invalid descendants immediately.

FINAL ONE-PAGE MANTRA

Evolutionary computation is natural biological generate-test search. The population evolves, not the individual. Fitness evaluates each candidate. Representation determines the valid operators. GA usually uses crossover strongly and mutation lightly; ES uses mutation centrally. Permutations require special operators. Parent selection chooses who reproduces; survivor selection chooses the next generation. Termination is mandatory. When MCQ keys conflict, learn the theory and mark the exam-risk expectation.

V3 provenance note

Supreme V3 keeps the full Supreme V2 theory and adds the strongest *examfinal.md* elements: compact quick-reference table, per-chapter exam handles, final-edition use protocol, ultra-high-yield priority order, and the V3 risk matrix. The over-absolute statements from examfinal were deliberately softened where theory and MCQ keys can conflict.